

NEMO day 2- ECR workshop transcript (12 May 2026)

00:00:03.846 --> 00:00:19.846

Inge Deschepper: Okay. Right. Hello, everybody. Welcome to the Canadian Ocean Modeling Early Career Day. We are very privileged to have Kirsten Meyer and then Andrew Shao today to give us the introduction to AI and the machine learning that we actually need, because I found yesterday for the AI talk, there were many words that I was like I don't think I understand this in the right context. So hopefully we can learn a lot from it. And if you have any questions, please put it in the chat. Please also, this is an encouraging inclusive environment, so please be respectful of everybody in the room. And if you are willing to, and you have your internet bandwidth, please keep your camera on. It does encourage a little bit more of, like, a community mindset, but please do not feel required to put it on. And then I think I will just hand it over to Kirsten and Andrew. I would you like to do self introductions?

00:01:15.797 --> 00:01:24.797

Andrew Shao: Sure. Yeah. Kirsten, you want to go? I think if folks were on yesterday, I introduced myself during that discussion. So, Kirsten, if you want to take it away for yourself.

00:01:24.712 --> 00:01:42.712

Kirsten Mayer: So hi everyone. I'm Kirsten Mayer. I am a scientist at the National Center for Atmospheric Research in Boulder, Colorado. I think for once, I'm actually on the same time zone as most of you on this call, the weird mountain time time zone. So that's really cool. Yes, nice.

Yeah. And basically my research has focused on how we can use machine learning in creative ways to understand our earth system better. I'm an atmospheric science by training, atmospheric scientist by training, not an ocean scientist.

So there's that, but I will try to focus my presentation here on some ocean applications for all of us. Um, very simple, but I hope that it ends up being useful. Great.

00:02:18.873 --> 00:02:20.873

Andrew Shao: As a quick reminder for people, so I am Andrew Shao, I'm in HPE, I used to be at CCCMA, and GFDL before then. Also visited NCAR a number of times, which is how I know Kirsten, cause you can't help running into Kirsten if you do AI and ML stuff at NCAR. Um, and yeah, so really glad that Kirsten was able to kind of join us today, so then again, my thanks to you Kirsten for taking the time to participate in this, and yeah. Kirsten is also lovely to work with, if you have any questions about anything please feel free- I'll volunteer her time to follow up with her, but yeah, she's an excellent resource to have just

to have a good discussion.

00:02:53.404 --> 00:03:13.239

Kirsten: That's totally fine. Yeah.

Andrew: Cool.

Kirsten: And with that said, I will say that I very much appreciate when people interrupt and ask questions during it. I feel like then I know that people are either following or not following at all. And so that's very, really helpful. And I try to be very respectful of all types of questions, so

00:03:14.180 --> 00:03:19.180

any question you have, I'm sure someone else has it, so feel free to ask.

Anything else you want to say, Andrew?

00:03:20.044 --> 00:03:23.044

Andrew: Nope, I think we're... I'm good to go from my side. Inge, you good?

00:03:24.114 --> 00:03:35.536

Inge: I think all good. I just remembered that if after the lecture portion, we could just do a group photo of everybody, like cameras on, but we can do that after the lecture.

00:03:36.727 --> 00:03:39.727

Kirsten: Okay, you want to do that after the lecture before the exercise?

00:03:40.133 --> 00:03:57.133

Kirsten: Okay, fantastic. Okay, so basically we've set up today in two portions. The first portion will be about an hour long, depending on how many questions you all have. And that's going to be focused on an introduction to artificial neural networks, which I would argue is one of the simplest machine learning

00:03:57.134 --> 00:04:14.158

algorithms there is out there. And then after that, we have also created a exercise where you get to train your own artificial neural network, and that'll be for the second hour. And I have some objectives for you all to try to accomplish using this

00:04:13.586 --> 00:04:29.586

tutorial or exercise. And if you finish that early, you're welcome to leave if you don't have any other remaining questions. Otherwise, feel free to stay the whole hour. I'll be here. Andrew will be here. We can answer any questions that you may have.

00:04:30.550 --> 00:04:33.550

Alright. So with that, I will share my screen.

00:04:39.860 --> 00:04:44.860

And then I need to shrink you all. I got a thumbs up. Perfect. Thank you.

00:04:46.220 --> 00:05:01.220

Okay. So, as I mentioned, this will be a brief introduction to artificial neural networks and training in general. I want to give credit to the Barnes group at Colorado State University, now at Boston University for creating this

00:05:02.555 --> 00:05:17.555

presentation and slides, and the exercise. This is only a portion of the tutorial we gave back in 2022, but if you're interested in learning more, there's also a link at the top of these slides to the Zenodo for more information.

00:05:17.469 --> 00:05:25.469

I'm also happy to share these slides after the presentation, so you can actually just click the link rather than trying to screenshot it now.

00:05:27.237 --> 00:05:42.237

Okay. So before we dive into artificial neural networks in particular, I wanted to give a bird's eye view of what we're talking about in terms of machine learning, and then where within this structure we actually find artificial neural networks.

00:05:42.522 --> 00:05:53.522

So machine learning has a variety of sub topics underneath it, and the 2 main ones that I'll just mention here are unsupervised learning and supervised learning.

00:05:53.457 --> 00:06:10.457

Unsupervised learning, you've probably actually heard about these techniques before, if not done them. We have dimensionality reduction techniques and clustering techniques. You can think of like self-organizing maps or k-means clustering. Those are all unsupervised learning techniques.

00:06:10.708 --> 00:06:26.708

On the other hand, we have supervised learning, and this is what I think that most people think of when they think of machine learning. And supervised learning means that you actually give your machine learning model a correct answer that it's trying to actually guess, whereas these unsupervised approaches,

00:06:26.414 --> 00:06:45.414

it's trying to learn your data itself. And then under supervised learning, there's these two types of prediction problems. We have the classification. This is the question, oh, is there a cat or a dog in this image? And then there's the regression type prediction problem, which is actually predicting a number.

00:06:46.309 --> 00:06:54.309

And under the supervised learning category is where we find artificial neural networks.

00:06:55.203 --> 00:07:08.203

Okay, so what is machine learning? The definition is the field of study that gives computers the ability to learn without being explicitly programmed. And when I first learned this definition, I thought that was

00:07:09.138 --> 00:07:24.138

crazily scary in terms of like computers learning without being programmed like what? But practically speaking, it's just a technique for fitting data, and it's not restricted to a single or set of simple mathematical equations.

00:07:25.762 --> 00:07:41.762

So, as I mentioned, the 2 types of machine learning that I'm talking about here- there are others- but just as an introduction, there's the unsupervised, the principal component analysis, clustering. This is looking for patterns within our data set with no labels that you already have.

00:07:42.819 --> 00:08:00.819

And then, where we're focusing today is the supervised learning, where we're basically having this machine learning algorithm map our input to our output that we have the answers to. So we'll have these input output pairs that we need to actually apply supervised machine learning.

00:08:00.548 --> 00:08:13.548

And so again, this is like regression and artificial neural networks. This also includes decision trees and random forests, which we won't be getting into today, but are another very simple application of machine learning.

00:08:16.372 --> 00:08:31.372

Okay, some reasons to use machine learning for science. If you don't know them already. Sometimes machine application, machine learning applications can do it better. Not always the case. But, for example, we have parameterizations within our Earth system models.

00:08:31.496 --> 00:08:43.496

We can use machine learning to maybe learn more expensive physical

representations of these certain processes, and that could make them more accurate.

00:08:43.849 --> 00:08:58.849

We could do it faster. In addition to these parameterizations, we have code within our models that are actually quite slow because they're actually trying to go through equations. If we use a machine learning model instead to learn these,

00:08:58.682 --> 00:09:06.682

these processes, these can be the machine learning model can, um, give you the answer a lot faster once it's trained.

00:09:08.202 --> 00:09:18.202

And then more related to what I do as a scientist here at NCAR is to try to use machine learning to learn something new about our Earth system.

00:09:19.010 --> 00:09:28.010

Um, basically using these tools, which are nonlinear, to look for relationships in our Earth system that we didn't know were there in the first place, because of the tools that we've had to date.

00:09:30.107 --> 00:09:49.107

In climate science in general, machine learning has made a huge impact, as I'm sure we, as I know you've mentioned, you talked about a lot yesterday in terms of machine learning being an important part of your group. And it's been an important part of, I would say, climate science and earth system science in general over the last

00:09:50.075 --> 00:10:11.075

close to a decade now. Ways that machine learning has made an impact is, as I've already mentioned, parameterizations and improving the speed up of particular models. Sometimes they can provide better predictions. These are like post-processing our forecasts or doing data-driven forecasts, especially on weather time scales.

00:10:11.754 --> 00:10:23.754

Um, my work focuses on sources of predictability, trying to understand our Earth system using machine learning, particularly applying explainable and interpretable artificial intelligence.

00:10:23.636 --> 00:10:34.636

Um, there's feature identification. Some people need to look at a satellite image and be able to identify the types of clouds in these images. Maybe identify extreme events, etc, etc.

00:10:34.554 --> 00:10:43.554

And then summarizing a lot of data. These are dimensionality reductions and clustering, as we've already mentioned, these more

unsupervised approaches.

00:10:45.632 --> 00:11:03.632

Okay, so now that we have like a general overview of where artificial neural networks fit within the space of machine learning, I want to just talk through the basic idea of what machine learning is, and particularly artificial neural networks. So the basic idea is to find this best function for fitting

00:11:03.484 --> 00:11:17.484

the input to the output. So we have some variables that we're inputting into this machine learning model, and we're asking it to produce some output. And in the process we're asking it to learn this nonlinear relationship between our input and our output variables.

00:11:19.721 --> 00:11:23.721

And I actually want to start all the way back with linear regression.

00:11:23.488 --> 00:11:29.488

Um, which is something you probably have learned recently.

00:11:29.301 --> 00:11:51.301

So. Input X and some output Y . If you know that X and Y are linearly related, then you know that you can model their relationship with $y = mx + b$, or this y equals A_0 plus A_1 times X here, where this A_0 is our y -intercept, and A_1 is the slope.

00:11:51.844 --> 00:11:54.844

And together they all lead to the prediction $\hat{y}$.

00:11:57.158 --> 00:12:12.158

Now, when we fit a line to our data, we're actually just trying to find an A_0 , our y -intercept, and a slope, A_1 that minimizes some sort of loss or cost function that we've defined. And typically that's

00:12:12.964 --> 00:12:22.964

something like mean squared error or mean absolute error. And so in this case, I've just included mean squared error, which is basically the difference of your prediction from your true value squared.

00:12:23.563 --> 00:12:35.563

And so we actually have a way to identify A_0 and A_1 , because we actually have an equation for it. But what about the cases when we don't have an equation for our data set?

00:12:35.743 --> 00:12:39.743

And that's where these artificial neural networks, or ANNs come in.

00:12:40.352 --> 00:12:55.352

So what I want to do now is step through how we can take our linear

regression that we just talked about briefly and put it into this diagram that I have a feeling that all of you have seen before, which is a bunch of circles connected by a bunch of lines that ultimately

00:12:55.992 --> 00:13:16.992

represent machine learning, or particularly artificial neural networks. Okay, so on the left-hand side of this screen, I'm gonna put in these circles and lines that are typically representative of an artificial neural network. And on the right hand side I'll have the equation being built up to create a linear regression.

00:13:18.045 --> 00:13:21.045

Okay, so let's say we have some input X one.

00:13:21.045 --> 00:13:33.631

Then we also will have this A_1 , our slope and our A naught. So A_1 will be multiplied by X , one. And then we're going to sum together this A naught term in this the center here.

00:13:36.149 --> 00:13:51.149

And so already we have a linear regression where we have our input. We have A_1 multiplied by our input, and we are summing together with our A naught. Now I just want to change these wording that I've been using into terminology that we use in the machine learning community.

00:13:50.913 --> 00:14:05.913

Um, because basically, um, at the fundamental level, I think you all understand machine learning. It's just we have a different terminology that we use. So instead of saying A_1 is our slope, we're actually going to call it a weight.

00:14:05.914 --> 00:14:11.185

And then this A naught term here, which is our y intercept. We're going to call this a bias term.

00:14:12.164 --> 00:14:22.164

Our output prediction, or our prediction, is just this equation that we have on the right-hand side. And then on the upper corner of this, um.

00:14:22.773 --> 00:14:26.773

On the screen here, we have our architecture, which is just our our full equation.

00:14:28.152 --> 00:14:39.152

Now we can create this for a multivariate linear regression case. So now we have X_1 and X_2 . These are both multiplied by their weights W_1 and W_2 .

00:14:39.910 --> 00:14:49.910

And then they're summed together in what we refer to as a node. You'll notice I've dropped off this bias term as these aren't typically demonstrated within these diagrams of artificial neural networks.

00:14:50.323 --> 00:14:59.323

Okay, so at this point, you're like, okay, this is just linear regression. Why do I even care? What makes artificial neural networks so awesome?

00:14:59.464 --> 00:15:14.464

Well, it's this, what's referred to as an activation function. And basically this activation function provides this nonlinear mapping of your linear algebra. So typical activation functions are sigmoid, tanh, ReLU.

00:15:14.843 --> 00:15:23.843

And the list goes on. But these are some of the most common ones that I see applied in artificial neural networks.

00:15:24.057 --> 00:15:43.057

Okay, so what do they do? So let's say we have an output of our node that's minus 2. So this x_1w_1 plus x_2w_2 plus b leads to a value of minus 2. So we pass that into our activation function. Let's say we've decided that for this node, our activation function is going to be ReLU.

00:15:43.913 --> 00:15:46.913

So, if we look over at the ReLU function here.

00:15:46.762 --> 00:15:58.762

It's max of $(0, X)$, meaning that anything greater than zero is just going to be... 0.

00:15:59.035 --> 00:16:16.035

The output of this node will be 0. On the other hand, if x_1w_1 plus x_2w_2 plus b leads to 2 instead of negative 2, the max of 0 and 2 is 2. And so we'll ultimately output 2 as the output of this node.

00:16:18.665 --> 00:16:33.665

So one way to think about an activation function is that it's basically telling you, telling the machine learning model whether a node is activated or not. And that's in the simplest case with ReLU. If it's a 0, it's not activated. If it's a value greater than 0, it is activated.

00:16:34.870 --> 00:16:41.870

Phrasing this another way, an activation function determines if information is moving forward from that specific node.

00:16:42.249 --> 00:16:54.249

And basically, this function, as I've already said, is this nonlinearity that is applied within artificial neural networks, and without it we would just be doing linear algebra.

00:16:56.650 --> 00:17:04.650

Okay, so that's very simple. We have one node, one input or 2 inputs. And we're trying to make a prediction.

00:17:04.337 --> 00:17:22.337

So machine learning or artificial neural networks can become complicated very quickly, because we can have multiple nodes within a hidden layer, which are these gray area, these gray layers in the middle here with multiple nodes. So we have 3 nodes in the 1st hidden layer and 2 nodes in the second hidden layer.

00:17:23.057 --> 00:17:37.057

Um, we can have many, many inputs. This case we have 3, but you can end up with thousands, which is the case that we're going to be doing today. And then you can have multiple predictions. So in this case there's 2, but you can also have thousands of predictions on the output as well.

00:17:38.934 --> 00:17:49.934

So when people refer to training in artificial neural network, all they mean is that they're trying to identify these weights and biases that we've already talked about that minimize some sort of loss function.

00:17:49.934 --> 00:18:02.324

So once you have a machine learning model that's trained, an artificial neural artificial neural network that's trained. You basically have an array of weights and biases that you can apply to new data.

00:18:04.735 --> 00:18:13.735

Alright, I've been speaking a lot, and I want to just talk through some terminology, and then I'm going to pause for questions to see if anyone has any.

00:18:15.112 --> 00:18:33.112

Okay? So what I've been referring to as nodes, those gray circles and the hidden layers can also be referred to as units or neurons. It's basically this computational unit connected by weights to other nodes. All that means is this is where we do the summation, and we apply our activation function.

00:18:34.125 --> 00:18:44.125

Weights and biases. Weights are the strength of the connection between the nodes. And we also have this bias term, which you can think of as like a shifting term, like your y-intercept.

00:18:45.164 --> 00:18:58.164

Then we have our activation functions. This is the nonlinearity of our artificial neural network. This is that function that maps the output of the node nonlinearly and determines whether our node is quote unquote activated.

00:18:58.666 --> 00:19:06.666

Then we have our hidden layers. These are layers of nodes. These are different from our input and our output layer.

00:19:06.989 --> 00:19:19.989

And then we what are typically referred to as deep learning is just machine learning, but training an artificial neural network with 3 or more layers. It's typically the accepted definition of deep.

00:19:24.211 --> 00:19:29.211

Okay, I want to pause quickly for questions, because I can't see anything.

00:19:29.961 --> 00:19:31.961

Grace Kirkpatrick: Um, yeah, I would love to ask a quick question.

00:19:31.961 --> 00:19:36.669

Kirsten: Yeah, go ahead, Grace. Um, so for the, um, activation function,

00:19:36.670 --> 00:19:38.394

Grace: Um, so for the, um, activation function,

00:19:38.395 --> 00:19:43.225

Um, I'm just assuming that this is kind of explicitly linked to the loss function, and that, like, a node would

00:19:43.691 --> 00:19:47.691

not be activated if it, like, perhaps exceeded some kind of, like,

00:19:49.109 --> 00:19:51.109

accuracy limit or something? Like, is that...

00:19:52.956 --> 00:20:10.956

Kirsten: Uh, no, actually. So the activation function is basically all that's required for this function is that it's differentiable, and I'll go into actually how the activation function is incorporated into the training process a little later on in in the actual slides, but

00:20:11.105 --> 00:20:20.105

the, there's no direct connection between the what loss function you use and the activation functions that you have.

00:20:20.105 --> 00:20:23.213

Grace: Okay, and I guess the other question I had was, um,

00:20:23.885 --> 00:20:34.885

Is the, um, distinction between, like, deep learning and being, like, 3 plus layers, is there, like, a qualitative difference in what happens, like, what happens at 3 plus layers, or is it just sort of an arbitrary distinction?

00:20:40.490 --> 00:20:44.490

Kirsten: I think it's arbitrary. Andrew, do you know?

00:20:44.490 --> 00:21:00.268

Andrew: So, I think that... the... the concept of it being a deep learning network, right? If you only have two layers, you have an input layer and an output layer, then there's kind of a direct transformation pathway. I would say the reason why 3 and more is kind of considered that is because you have that one hidden layer.

00:21:00.981 --> 00:21:21.981

And that hidden layer is where, you know, the... it's... Not necessarily interpretable. It becomes really difficult to interpret, right? Because that's kind of like the r... not random, but the heuristic understanding, the heuristic learnings are stored in that hidden layer. So I think that there, especially in the context of deep learning, when we start getting to these, like,

00:21:21.981 --> 00:21:38.870

much more complicated architectures. This kind of relates to a question yesterday that somebody had asking about what do you mean by a latent space? And so that latent space, this, like, reduced understanding or this reduced basis that is being learned by the machine learning network

00:21:38.915 --> 00:21:53.915

can only really exist if you have kind of this one hidden layer in between, at least one hidden layer in between. But yeah, other than that, that's my best understanding as to why Deep is there, because it allows for this kind of one hidden layer.

00:21:53.916 --> 00:21:58.295

As opposed to just a direct input and then final transform.

00:22:02.109 --> 00:22:07.109

Kirsten: Thanks, Andrew. Any other questions?

00:22:14.062 --> 00:22:17.062

Andrew: Amber, I think you're on double mute?

00:22:17.062 --> 00:22:23.158

Still can't hear you. Yeah.

00:22:23.159 --> 00:22:29.351

Yeah, you're good. Yeah.

00:22:29.352 --> 00:22:42.687

Amber Holdsworth: How about now? How about now? Okay, a wrong input. Yeah, so while we're on the questions of terminology, you actually didn't talk about hyperparameters, but I just, while we're talking about it, in this case, I just want to make sure my understanding is correct and it might not be. So when you're talking about hyperparameters in this case, could we say like,

00:22:43.120 --> 00:23:03.120

the linear regression function is our, like, chosen function. And then like it's a polynomial, and we've chosen a hyperparameter of one. So it's like x to the 1 or whatever, and in this case, that would be, like, our really simple, like, hyperparameter, because I thought I think hyperparameters are, like,

00:23:03.887 --> 00:23:08.887

part of the functional form that you use, but it isn't trained by the model. Is that correct?

00:23:19.483 --> 00:23:21.483

Kirsten: So yeah.

Amber: Okay.

00:23:21.483 --> 00:23:28.527

Kirsten: So basically hyperparameters are things like how many nodes you want in a layer, how many layers you want? What activation functions you use, what loss function, what your learning rate is. These are all things you need to define in order to train your machine learning model, and I'll get into more of those

00:23:28.389 --> 00:23:33.389

parameters that you need to specify later on. Does that make sense?

00:23:33.390 --> 00:23:38.308

Amber: Okay. Yeah, thanks.

00:23:38.309 --> 00:23:39.477

Kirsten: Okay.

00:23:39.477 --> 00:23:42.839

And then, is there there any other questions? Possibly people double muted.

00:23:45.013 --> 00:23:49.013

Okay, no. All right. Then I'll then I'll just get. I'll continue.

00:23:49.013 --> 00:23:56.195

Okay, and feel free to. I you can interrupt me even if I'm not pausing for questions. So just feel free to go. Go ahead and and start speaking.

00:23:57.881 --> 00:24:05.881

Okay, so now we're going to go into this question of what do I mean when I say best function for fitting your input to your output?

00:24:05.882 --> 00:24:12.850

And in order to start talking about defining best, we need to go all the way back to talking about our data.

00:24:12.929 --> 00:24:27.929

So typically in machine learning applications, we split our data into 3 data sets. So we have our full data. Then we have from our all of the data, we have a training data set, which is the majority of your data typically, our validation data set,

00:24:27.763 --> 00:24:35.763

and our testing data set. Now, I want you to forget about the testing data set at this point. We're just going to talk about the training data and the validation data.

00:24:35.713 --> 00:24:48.713

So when we are training our model, we're only going to be looking at our training and validation data. So the training data is what we're going to use to actually determine these weights and biases in our model. This is how we build our machine learning model.

00:24:50.193 --> 00:25:14.193

While we're training, we apply our validation data to see how the machine learning model is performing on unseen data, data that it's not being used- data that's not being used to train the model. We can also use this validation data to determine what hyperparameters we would like to use for our machine learning model. So let's say you have a network with 3 layers and a network with

00:25:13.625 --> 00:25:27.625

one layer and your hidden layer, and you're wondering which one will perform better for your specific application. This is where you would use... you'd train your model using your training data, and then you'd evaluate its performance on your validation data.

00:25:27.427 --> 00:25:44.427

Now, once you've chosen what machine learning architecture you'd like to use, and that you think it performs well on your training and validation data, that's when this testing data is going to come back

in, and that's what you actually fully evaluate your machine learning model on.

00:25:48.732 --> 00:26:11.732

So typically in the machine learning space, you'll often see that we apply random splits. And if as our system scientists, you know that there is autocorrelation, temporally and spatially within our data sets. And so randomly splitting our data into trainee validation and testing data isn't necessarily the best idea for most of our applications.

00:26:11.710 --> 00:26:22.710

So generally, what people do are chronological splits, meaning that they take certain sections of our data, all

00:26:22.486 --> 00:26:30.486

between some years, for example, and that'll be our training data. Then the next for validation, and the next for testing, for example.

00:26:31.648 --> 00:26:36.648

And, as I've already mentioned, this is because our data is auto autocorrelated. So.

00:26:36.594 --> 00:26:52.594

For example, let's say that we have a really strong El Nino, and we've decided to split our training and validation data on that very strong El Nino. Now you have that event in your training data, and you also have it in your validation data. So when you

00:26:52.447 --> 00:27:06.447

apply when you train your model and you evaluate it, the performance on the training data, and you apply it on your validation data, you may be getting inflated skill because your training data also had the exact same event within the validation data.

00:27:06.822 --> 00:27:13.822

And then when you go to test your machine learning model, it may perform a lot worse. And that's in an extreme case.

00:27:13.951 --> 00:27:22.951

Oftentimes, you probably won't even notice that this has happened, but it's important to know when you're actually dividing your data to account for things like this.

00:27:22.466 --> 00:27:38.466

Sometimes people actually leave a certain gap between their training, validation and testing data set so that they feel that they've completely separated each of these. Something that's very important to know in machine learning, as these are very data-hungry

00:27:38.989 --> 00:27:49.989

algorithms, so it's best to keep as much data as possible, but it's also very important not to leak information between your training validation and testing data sets, that they're quote-unquote independent.

00:27:49.990 --> 00:27:55.287

So there's a balance that you need to strike in terms of those decisions that you're making.

00:27:57.320 --> 00:28:16.320

Okay, so why do we split our data in this way for this quote unquote best function. So I'm going to step through this example here. So we have testing data in this black color, training in gray, and validation in blue. And let's say we only have one input X, and we're trying to predict our y.

00:28:16.634 --> 00:28:22.634

The solution that we end up finding that represents our data well is in this gray line here.

00:28:22.323 --> 00:28:31.323

And this is a good example of your solution representing the training data set, the validation data set, and your testing data set well.

00:28:32.210 --> 00:28:47.210

On the other hand, if you quote over-fit to your data, your training data in particular, this might be what your machine learning model has learned. It's particularly focused on these gray dots, and it's ignoring a lot of the structure of the

00:28:47.135 --> 00:29:02.135

blue and the black dots. So you're going to perform really well on your training data set. In this case, probably 100% accuracy based on this line. But on the other hand, you're going to perform way worse than that in terms of your validation, and way worse in terms of your testing.

00:29:03.695 --> 00:29:11.695

So too closely fitting the training data such that the model will fail on unseen data is what we refer to as overfitting.

00:29:12.088 --> 00:29:22.088

and basically what we're trying to accomplish by training our machine learning model with these training and validation sets is to make sure that we don't overfit.

00:29:25.052 --> 00:29:34.052

Okay, I'll get back to overfitting once I talk through some additional topics that we need to learn to actually explore overfitting further.

00:29:34.052 --> 00:29:50.492

So how do we quantify a good prediction? This is where this cost slash loss function comes back in, and we've already talked about it a decent amount. So we have our mean squared error here as an example. And basically, you need to define what cost or loss function you want to use.

00:29:50.553 --> 00:29:58.553

Um, and basically what you're going to try to do is minimize that loss function to help determine what your weights and biases will be for your machine learning model.

00:29:59.442 --> 00:30:07.442

So how do we actually find the minima in a real life example? So I'm going to step. I'm going to go back to this linear example here, where we have y equals mx plus b .

00:30:07.303 --> 00:30:23.303

On the far right-hand side, we have our... we're fitting a line to our values, which are in blue here. We have X on the x-axis and our target T on the y-axis. And for this, for simplicity, so that we only have a 1d example here.

00:30:23.678 --> 00:30:37.678

We're going to say that our y-intercept is zero. So we're all we're trying to find is our $A1$ value or P in this example. So P or $A1$ is plotted on the x-axis here, and our loss function is plotted on the y-axis.

00:30:37.902 --> 00:30:47.902

And what you see is that as you update your parameter P , you move around in this loss space to ultimately find the minima of your loss function.

00:30:50.941 --> 00:30:58.941

Now, in order to do this in the machine learning case, we call this gradient descent. We're basically trying to find the

00:30:58.903 --> 00:31:14.903

direction of greatest decrease. So if you remember back to calculus, a gradient gives you the direction of greatest or fastest increase, and so this descent here just means that we want to follow the greatest decrease, and it's just represented by a negative value.

00:31:16.312 --> 00:31:34.312

So in order to do gradient descent, there's a hyperparameter that you have to account for, and this is referred to as the learning rate. If your learning rate, how big your step is that you move in your loss space, if that is too small, it could take you a very long time to

converge at your local minima

00:31:34.201 --> 00:31:42.201

or global minima, hopefully, but you'll never know. So here's the an example of this very small step.

00:31:42.980 --> 00:31:48.980

On the other hand, if you have a really high learning rate, your step could be too big, and you could completely overshoot your minima.

00:31:48.980 --> 00:32:05.492

So this is a bit of a Goldilocks scenario in trying to find a decent learning rate that fits for your application. And I'll give you some classical examples of what a learning rate value is. But there's also methods that people apply to help address this problem of very small versus very large.

00:32:05.259 --> 00:32:21.259

One example is that people have learning rates that decrease throughout training, so they adapt to your training process. So you start with a very large learning rate and then it slowly becomes smaller throughout the process. That's one example of how you can adjust your learning rate during

00:32:22.107 --> 00:32:29.107

your training process. So gradient descent is our method that we use to minimize our loss in cost function.

00:32:29.485 --> 00:32:34.485

and then that learning rate is how quickly we're going to try to minimize that loss function.

00:32:36.372 --> 00:32:51.372

So mathematically, let's go back to our linear example here. So let's say mean squared error is our loss function, as we've already talked about. This is going to depend the value of our loss is going to depend on this A_{naught} and A_1 .

00:32:51.517 --> 00:33:00.517

And so in order to calculate our gradient, we're just going to take the partial derivatives of the loss function with respect to A_{naught} and A_1 , or these weights.

00:33:00.788 --> 00:33:08.788

And that's what's shown in the bottom right-hand side. Here we have the gradient of our loss function with respect to A_{naught} and A_1 .

00:33:09.495 --> 00:33:15.495

I apologize for all the math on this screen. But just very briefly.

00:33:15.897 --> 00:33:31.897

We can actually calculate these gradients when we know what the functional form or the equation of our prediction is. So in this case, if we have our mean squared error, which is right here, we have our prediction minus our actual value squared.

00:33:32.997 --> 00:33:39.997

We can put in for our prediction the equation that we have. So that's A_0 plus A_1 times X .

00:33:39.998 --> 00:33:53.274

Then we can take the derivative in respect to A_0 . And that is, and this is what we end up getting on the right hand side here. We can also take the derivative in respect to A_1 , and we get this equation on the right hand side here.

00:33:55.051 --> 00:34:09.051

Then, once we've taken the derivative, we can multiply it by this learning rate parameter λ , and we basically take our original A_0 value, and we subtract it from this learning rate times our derivative.

00:34:08.282 --> 00:34:19.282

And we do the same for A_1 . And those provide us our new weights for our machine learning model that moves in the direction of fastest decrease, hence the negative in this case.

00:34:22.241 --> 00:34:31.241

Okay. So you may have noticed that I said we can do this if we actually know the equation of our of our application.

00:34:31.862 --> 00:34:45.862

However, the whole part of point of an artificial neural network is you don't need to know what your equation is. You don't need to know whether it's linear or polynomial, or what order of a polynomial it is. This is what your ANN is trying to figure out for you.

00:34:48.065 --> 00:35:05.065

So the way that we approach this problem. Hang on. You guys are in my way here. Okay, perfect. The way we approach this problem is through back propagation. Which is short for back propagation of errors. It's how gradients are calculated for gradient descent in a machine learning framework.

00:35:05.782 --> 00:35:22.782

So this is the equation we saw previously for A_0 and A_1 . Now I just have w_k here instead. And so we can put our loss function in here. But now, when we actually calculate the gradient with respect to that particular weight of our loss function here,

00:35:22.380 --> 00:35:32.380

we end up with these $\hat{Y}$ predicted terms. So now we need to find out a way to actually calculate these $\hat{Y}$ predicted terms.

00:35:32.428 --> 00:35:41.428

and I'm not going to go into the details here. But the the short answer here is that we use the chain rule.

00:35:41.612 --> 00:35:59.612

and I would actually highly recommend that you look into this 3 blue, one brown neural network series. I feel like I should get paid for how much I sponsor this series. Video is chapter 4. He does a fantastic job of explaining

00:35:59.426 --> 00:36:16.426

how back propagation works in the case of an artificial neural network better than I would ever do. And I don't have the time to go into this here. But if you don't learn well from people speaking at you, and you'd rather read about it, there's also this free book

00:36:17.112 --> 00:36:24.112

that I found on the web, um, neuralnetworksanddeeplearning.com chapter two. So you can also check that out as well.

00:36:25.557 --> 00:36:28.557

Okay, do we have any questions about this so far?

00:36:42.932 --> 00:37:00.932

Okay. So there are a variety of different gradient descent algorithms that we can use in the exercise that I have provided. I give 2 options. So we'll have the option to use SGD or Ada, and this just affects how you move around in your loss space

00:37:00.932 --> 00:37:05.519

basically, based on this diagram here, as you can kind of see an example.

00:37:07.418 --> 00:37:19.418

Okay, so taking now that we've really got ourselves into the the math, I want to take a step back again and talk about the machine learning approach. So how do we actually train our machine learning model?

00:37:19.486 --> 00:37:40.486

So we have some sort of training data, which is our predictors. These are passed into this function that we're trying to learn, and we make a prediction. That prediction is compared to our truth using back propagation. We then pass it back into our machine learning model and update our weights and biases so that they minimize our loss function or move in the direction of decrease.

00:37:40.425 --> 00:37:53.425

Then we give our machine learning model more data. We make more predictions. We compare it to our truths, and this cycle just continues and continues and continues. And that's all that the training process is.

00:37:56.230 --> 00:38:01.230

Something that we need to talk about when we talk about training is batch size and epoch.

00:38:02.104 --> 00:38:26.104

So typically for memory constraints of GPUs, we typically chunk our data into smaller sections or batches of our training data. And basically what that means is that we take a subsection of our training data, we pass all of those samples through the machine learning model before we update the weights and biases. So we basically take the average performance of all of those samples and use that to

00:38:27.065 --> 00:38:38.065

calculate what weights and biases we should have. And then in epoch, or epic, whichever one you prefer, happens when you do one complete pass through the entire training data set.

00:38:40.183 --> 00:38:50.183

So typically you train for multiple epochs, meaning that you see your training data set more than one time, and this is how you ultimately minimize your your loss.

00:38:50.597 --> 00:39:06.597

So to give an example of batches and epochs here. There's one at the bottom. Let's say our sample size is 100. This is way too small for our machine learning application. But it's a very simple way to approach this. So let's say your samples are 100, and your batch size is 20.

00:39:07.136 --> 00:39:17.136

You will have 5 batches of 20 samples to get to your full 100 samples of your training data set. And once you go through those 5 batches, you've gone through one epoch of training.

00:39:21.042 --> 00:39:33.042

Okay, so I wanted to jump back to overfitting here, and I want to talk about how we can actually identify over or one way we can identify overfitting when we're training our machine learning models.

00:39:33.455 --> 00:39:37.455

And so I actually have a question for all of you.

00:39:37.885 --> 00:39:54.885

On the x-axis here we have training time, which is the number of times

we go through our training data set. And on the y-axis we have our loss metric. In the gray line, this is an ideal curve for your trading data in terms of minimizing your loss metric.

00:39:55.857 --> 00:40:15.857

and I want to ask what you guys would think the validation data would look like if we were actually overfitting throughout this training time process. So I've given the 1st little bit seems to be moving with the the training data. And now what do you think will happen if now our machine learning model starts to overfit to the training data.

00:40:38.056 --> 00:40:51.056

Inge: Well, if I'll give it a try. To me, I guess if it overfits, then it will just continue down rather than actually follow the curve like just more good rather than.

00:40:55.827 --> 00:40:57.827

If it's over...

00:40:57.828 --> 00:40:58.062

Kirsten: Do you think the purple line would continue downwards like this? Oops. Am I giving the answer?

00:40:58.566 --> 00:41:07.566

Inge: Well, almost like crossing the line, like... instead of curving. Yeah, curving out. Yeah, something like that. Yeah.

00:41:07.566 --> 00:41:13.661

Kirsten: like this. So the hope of our machine learning model is that we're trying to minimize this loss. So an ideal case is that we have the.

00:41:13.356 --> 00:41:20.356

our loss value goes to zero for both our training and validation. So that would be the hope.

00:41:24.761 --> 00:41:26.761

So if we're... Yeah.

00:41:26.762 --> 00:41:29.256

Inge: Oh, okay, now I understand. Okay, so...

00:41:29.257 --> 00:41:34.803

Amber: So it... So you're so you're asking. Okay, so the validation. This is when so we've trained the model, and now we're applying it to like this unseen data set, but it's overfit on our training set. And that's so that's...

00:41:38.388 --> 00:41:46.388

Is that the idea here? And so then this would be... it should be wrong.

Like, I mean, you already showed it, but...

00:41:54.223 --> 00:41:55.523

Kirsten: Yeah, great. So yeah, yeah. So actually, the validation data is what we evaluate during the training process.

Amber: Oh, right.

Kirsten: So we'll we will actually see the validation throughout the training process. So a typical diagram that you'll see, and you'll have it in the exercise, too.

00:42:09.110 --> 00:42:14.137

is, um, two curves of your loss, and one is on the training data, and one is on the validation data.

Amber: Okay.

Kirsten: But all the data that's being used to update the weights is your training data, and then the validation data is this other data set that you can say, oh, is it still performing well on unseen data?

00:42:15.722 --> 00:42:20.722

Amber: Right. And then so then you would expect your loss metric to

00:42:20.605 --> 00:42:27.605

be really good for the training data, and then once you apply it to validation, it's gonna suck. Yeah.

00:42:33.976 --> 00:42:34.403

Kirsten: Exactly. Yes. And I apologize for showing the answer before you guys got it.

00:42:38.837 --> 00:42:42.837

Amber: I was still processing what's happening. Thanks.

00:42:42.837 --> 00:42:56.815

Yeah, yeah. Yeah, thanks for walking it through. Okay, so yeah, so if our machine learning model is overfit, we would expect it to do really well on our training data set. And then as it becomes more and more overfit, we would expect it to perform worse and worse on our validation data set, which is why it ends up going up.

00:42:59.855 --> 00:43:01.855

Okay, do we have any more questions on that?

00:43:05.679 --> 00:43:12.679

Amber: Yeah, so but this is just a simplistic example. Again, typically you would do like cross validation.

00:43:16.831 --> 00:43:18.831

Right. Yeah.

00:43:18.831 --> 00:43:24.859

Kirsten: Yes. Yeah. cross validation is also a great way to, especially if you don't have a very large data set, how well your machine learning model is performing. Yeah.

00:43:30.782 --> 00:43:41.782

Okay, so let's say you saw this loss curve while you were training, and you wanted to try to figure out a way to address this overfitting. There's a couple ways to do this.

00:43:41.204 --> 00:43:58.204

And the the first 2 I'm going to show are basically penalize adding additional penalization to your loss function. So let's say you have your mean squared error. This just given the screen grab that I got, it says root mean squared error. But let's say it's just mean squared error.

00:43:58.563 --> 00:44:15.563

One way to try to prevent overfitting is using ridge regression or L2. Basically, this function forces your coefficients to share weights. This is can be particularly useful in ocean cases, because we have the spatial,

00:44:15.701 --> 00:44:23.701

um, autocorrelation, so you... you want your machine learning model to explore the space rather than to focus on one specific point in your input.

00:44:23.948 --> 00:44:30.948

Um, and so that's what this term does, is forces the machine learning model to share weights across inputs.

00:44:31.999 --> 00:44:42.999

On the other hand, we have lasso regression. Instead of sharing weights, this one is saying, if you have too many inputs, and some of them aren't useful, I'm going to force them to 0.

00:44:43.252 --> 00:44:54.252

So basically, this is helpful for determining which of your input is most important and getting rid of anything extra that could be causing your model to overfit to noise.

00:44:55.556 --> 00:45:11.556

And then the last one which I think makes a lot of sense in terms of a way to address overfitting is called dropout. This is this only occurs during your actual training process. So during validation and testing, you don't drop out

00:45:11.685 --> 00:45:20.685

specific nodes. But what happens here is, while you're training, you actually remove specific nodes in your machine learning model.

00:45:21.913 --> 00:45:45.913

And this means that your machine learning model is forced to make predictions with fewer nodes than it originally have had. So it's making the machine learning model share responsibility for a specific prediction. And some of these typical values of how much to drop out is around like 50 to 80%. So it's an extreme amount of nodes that are dropped out from your machine learning model and forcing it to actually make

00:45:45.914 --> 00:45:50.143

the same prediction that it would with all the nodes available.

00:45:50.845 --> 00:45:56.845

A downside of dropout is that it typically requires more epochs during training.

00:46:01.145 --> 00:46:18.145

Okay. So lastly, I wanted to talk about the softmax activation function that's typically applied at the output of ANNs that make a categorical prediction. And the reason I'm bringing this up is because we are actually going to do a categorical prediction problem.

00:46:18.522 --> 00:46:36.522

So in terms of a categorical prediction problem, that output layer applies the softmax activation function. And what this does is it converts raw model output into values that sum to one. So you can kind of think of it like a probability of that specific prediction. So in this case,

00:46:36.687 --> 00:46:43.687

this machine learning model has three classes. It's predicted a 5, a 2.5, and a 0.5.

00:46:43.895 --> 00:46:51.895

Then the these values are passed through the softmax function, and we end up getting 0.9, 0.1, and 0.0.

00:46:52.742 --> 00:47:07.742

Now, the machine learning model makes a decision on which the which class it that is chosen based on the biggest number here. So in terms of this prediction problem, the class that's predicted is this 0.9 class, or this top class here.

00:47:11.416 --> 00:47:20.416

All right. So in terms of ANN considerations, there's a lot we have to

determine. And this goes back to our hyperparameter discussion.

00:47:20.234 --> 00:47:42.234

So we need to figure out how long we want to train our model. These epochs, how many times we want to go through our training data set. If we don't iterate enough, our model won't have enough time to find an optimal fit. If we iterate too much, we could overfit to our training data set. We could start learning noise and then perform really poorly on our validation data set and testing data set.

00:47:42.601 --> 00:47:58.601

We need to choose the number of layers and nodes which construct our architecture. If you have a ton of layers, it can be very difficult to interpret; if you have very few layers, as Andrew was saying earlier, this can make it a little bit easier to interpret.

00:48:00.081 --> 00:48:03.081

The more layers, the more nonlinearities you can account for.

00:48:03.175 --> 00:48:12.175

You need to choose activation functions for your specific layers. And you need to identify a learning rate for your application.

00:48:12.985 --> 00:48:21.985

Again, if your learning rate is too large, you could skip your local minima, and if it's too small, it could take you a very, very long time to converge to that minima.

00:48:22.481 --> 00:48:34.481

Other things that we've talked about that you'll need to determine are what batch size you want to do, what size chunk of your training data would you like the machine learning model to look at before it updates its weights?

00:48:34.382 --> 00:48:46.382

Um, what gradient descent algorithm to use, whether you want to apply some sort of regularization, and then how you want to ultimately prepare and split your data or standardize your data, etc.

00:48:48.800 --> 00:49:05.800

Um, before we get started, I wanted to provide some typical values for some of these things. A learning rate is typically on the order of 10^{-2} , but can be very small depending on how much you're decreasing your learning rate throughout the training process.

00:49:07.803 --> 00:49:24.803

Batch size can be 32, 64, 128. They're typically of these factors of two, I think I was told at one point that this is for computational efficiency. Okay, I'm getting a nod, yes. So typically we get 8, 16.

00:49:24.776 --> 00:49:39.776

32, 64, 128, 256, 512, and it keeps going and going. Um, this is similar for the number of nodes. You can have four nodes, 8 nodes, 16 nodes. But you can do nodes that are not

00:49:39.061 --> 00:49:47.061

of these values. So you could do nodes of 10, nodes of 5. It's not it's not unheard of if you wanted to do that.

00:49:50.137 --> 00:50:04.137

Okay, so I'm just going to briefly talk through the exercise example that we're going to do next. But before I do that, I wanted to pause one more time to see if we have any more questions.

00:50:17.376 --> 00:50:20.376

Okay. Oh, yes, yes, go ahead.

00:50:20.376 --> 00:50:26.166

?: I had a quick question. Um, thank you for the presentation, firstly. I was curious, you mentioned at the start that you're interested in prediction and explainability.

00:50:26.560 --> 00:50:40.560

So what part of the kind of methods you mentioned are you using for that explainability piece? Are you kind of looking at which nodes are being dropped out, the weights or activation of the nodes?

00:50:40.981 --> 00:50:46.981

There can be quite a few weights, depending on, like, the hyperparameters. Like, what's your go-to thing to understand

00:50:46.420 --> 00:50:52.420

the explainability and... That latent space that is finding super important?

00:50:52.141 --> 00:51:08.141

Kirsten: That's a great question. Yeah. So I actually haven't talked about any explainability techniques here. And just to mention that the dropout is done randomly. It's not as specific thing that's learned in terms of which nodes to drop out just just to

00:51:08.254 --> 00:51:33.254

make that a little clearer. In terms of explainability, typical methods that I use are ones that look at the gradient of, so how much the output changes in respect to changes in the input. And so these methods include input times gradient, integrated gradients, and these are all methods that are applied after you've trained your machine learning model, and they explore

00:51:33.643 --> 00:51:37.643

this sensitivity of your machine learning model to changes.

00:51:46.147 --> 00:51:48.147

Cool. Great.

00:51:50.575 --> 00:51:55.575

All right. Oh, yep. Go ahead.

00:51:55.576 --> 00:52:09.645

Andrew: So just one. Sorry. So just one comment that I'll object just based on some of the conversations that were happening yesterday's discussion is again, like, you know, for those of you who have a background in statistical modeling, and that includes things like linear regression or any of these kind of regression techniques that you may have seen in undergraduate during graduate school

00:52:09.943 --> 00:52:26.943

you know, there... all those lessons that you learn when you do that statistical modeling, and you do those statistical techniques still apply to machine learning. You know, for example, right, one of the things that we know for linear regression is exactly what Kirsten was saying, is that it's really helpful to build robust.

00:52:26.654 --> 00:52:42.654

robust statistical relationships when you have more data. And one of the things, though, that is a cautionary thing, and again, goes to this question of regularization that Kirsten was talking about, is fundamentally, when we're doing a regression technique, we want to have an over-determined system.

00:52:42.860 --> 00:53:00.860

Right? And so what that means is that we really want to have more independent samples than we have number of free parameters of the statistical model. So you know, this is a trap that we see all the time for people who are starting to go into machine learning, where they're like, Oh, I can train 100 billion parameter model.

00:53:00.394 --> 00:53:09.394

Right? And you can do that nowadays, but that also means that you have 100 billion, or you have a billion free parameters, and so the question is,

00:53:09.887 --> 00:53:28.887

do you actually have an over-determined system? Do you have more data than you really want? And you know, like, good, like, you know, the same good rules of thumb that apply for statistical modeling again apply right? So if, like, you think of just how big your network is, and you probably need at the very minimum 10 times the number of independent samples.

00:53:28.887 --> 00:53:46.765

As the number of parameters, if not more, right? So, like, that's one of the key things to keep in mind. Like, bigger is not always better, and especially with machine learning models, there is this tendency to just kind of use the biggest, best thing that you can find. But statistically speaking, all those lessons, all those cautionary tales still apply

00:53:46.347 --> 00:54:02.347

to machine learning. So I would, I hope that one of the things that you're getting from this, and one of the reasons why I actually really liked Kirsten's slides in this, is that there's a lot of machine learning that is just an extension on some of these things, on these mathematical techniques that you might know. So,

00:54:02.233 --> 00:54:28.233

you know, this question of learning is really an optimization problem. This question of how do you define your network is really just what is your statistical model that you're using? So... you know, it's less scary than it sounds, and a lot of it is terminology and just understanding where are these interesting extensions are from like time honored techniques that we're just able to do now because we have bigger compute.

00:54:29.403 --> 00:54:43.403

Kirsten: Yeah, that's fantastic. Yeah, thanks, Andrew. And I'll just mention quickly that I actually have a slide a little further down that shows you actually how to look at the number of learnable parameters. And so you can do that when you're actually training your model in in a second.

00:54:45.432 --> 00:54:51.432

Inge: The other. Yeah.

00:54:51.433 --> 00:54:53.005

Andrew: Um, there's a question in the chat. Sorry.

00:54:53.005 --> 00:54:59.142

Yeah, Enrico says, how do ANNs take into account errors/uncertainty in the input data?

00:55:00.403 --> 00:55:11.403

Kirsten: Uh... I that they don't natively take in any information about that. You would have to provide some sort of

00:55:15.222 --> 00:55:19.222

information to the machine learning model around that.

00:55:19.223 --> 00:55:22.226

Dhamma K: I think on the errors in the input data, you can think of it

as like those,

00:55:22.241 --> 00:55:37.241

if you go back to the overfitting slide, you can overfit to errors, right? Yeah. So you could overfit to that bottom right gray dot, or you could regularize enough

00:55:37.304 --> 00:55:48.304

either by their architecture or the training or the loss function, and then, like, recover the actual relationship rather than overfitting to these errors in the training data.

00:55:52.940 --> 00:55:55.940

Amber: Oh, can you explain regularization?

00:55:55.941 --> 00:56:07.214

Dhamma: That makes sense. Uh, so regularization is things you do to prevent overfitting. So like what Kirsten showed and what Andrew was talking about when talking about the number of parameters versus the number of independent samples.

00:56:08.640 --> 00:56:13.640

There's a lot of things that go into it. Yeah, a lot of ways to regularize.

00:56:17.497 --> 00:56:23.497

Andrew: And Dhamma, since you unmuted yourself, do you want to introduce yourself since you didn't get to?

00:56:23.498 --> 00:56:31.330

Dhamma: Oh, yeah. And I'm Dhamma, I'm trained as a machine learning person. But I've been working at NCAR for 3 years.

00:56:35.644 --> 00:56:38.644

Uh, mostly on weather applications.

00:56:38.645 --> 00:56:41.535

Kirsten: And Dhamma actually has a degree in computer science, unlike myself.

00:56:41.536 --> 00:56:41.844

Andrew: And me.

00:56:42.041 --> 00:56:46.041

Kirsten: So yeah, great. Yes.

00:56:46.041 --> 00:57:02.560

Andrew: Yeah. So yeah, Dhamma is the third musketeer. He'll be helping out for in the sessions, doing things exactly what he did, monitoring the chat, responding to some chat questions in line. So feel free to.

And we again encourage people to post questions in the open public chat.

00:57:17.557 --> 00:57:20.714

So everyone can see the questions, everyone can see the answers. You know, this is definitely the space of all questions are good.

Kirsten: Absolutely.

Andrew: You know, there's a lot of confusion, there's a lot of overloading of terms, there's a lot of concepts. So feel free to post whatever and whenever, and, you know, we'll make sure that we capture the chat log

00:57:20.724 --> 00:57:24.724

as part of an artifact for of this presentation.

00:57:26.674 --> 00:57:30.674

Kirsten: Inge, did you want to say something? I saw you unmute a couple of times.

Inge: Oh no, that was just for Enrico's question, that's all.

00:57:36.829 --> 00:57:39.829

Kirsten: Perfect, fantastic. Okay.

00:57:39.134 --> 00:57:43.134

Okay, so now I want to talk through our exercise.

00:57:43.261 --> 00:57:56.261

So basically, what we're going to do here is we're going to input some tropical sea surface temperatures, and we're going to ask our machine learning model to determine whether there's a La Nina or an El Nino in that specific image of tropical SSTs.

00:57:57.530 --> 00:58:11.530

I'm not going to go through what El Nino and La Nina are in this space. I feel like that is quite redundant. But as a reminder, very briefly, La Nina is cold SSTs and El Nino is very warm SSTs in the tropical Pacific.

00:58:11.752 --> 00:58:29.752

Okay. Why do we care about? And so beyond the ocean is it affects where we live everywhere in the globe in terms of precipitation and temperature anomalies, and it can also help in terms of improving our prediction skill on a variety of timescales.

00:58:30.157 --> 00:58:47.157

Okay. So in terms of defining El Nino and La Nina, I'm using the Nino 3.4 index here. Again, you probably all know this, but it's located

between the Nino 4 and the Nino 3 regions, and it's defined in the in this box here.

00:58:48.817 --> 00:59:03.817

Um, based on that, this is what the Nino 3.4 index looks like. We have what I've determined for an initial start of our analysis is I've determined that a La Nina is less than negative 0.5, and an El Nino is greater than 0.5.

00:59:03.110 --> 00:59:21.110

This is a parameter you can change. Just a reminder that the higher you set your value in terms of the cutoff, that means that the fewer samples you're going to have as an El Nino or a La Nina. So just a reminder, but if you go too close to zero, you're not going to be able to identify.

00:59:22.007 --> 00:59:30.007

Well, I actually don't know, I haven't tried it. I don't know if a 0 will work in terms of determining an El Nino versus a La Nina. So maybe one of you can try that for me.

00:59:32.044 --> 00:59:44.044

Okay, as I mentioned, we're going to do a categorical prediction problem. So all that means is that all the El Ninos will be a one in your data set, and all the La Ninas will be set to 0.

00:59:44.812 --> 00:59:54.812

I've also colored in here the values that I've originally selected for training data, validation data in red and testing data in in orange.

00:59:54.773 --> 01:00:09.773

Um, and then something else to note here is I've removed all non El Nino and La Nina's from this prediction problem. So it's only El Ninos and La Ninas that you have as input based on your definition of an El Nino and a La Nina.

01:00:10.593 --> 01:00:17.593

Okay, so basically, you're going to have SSTs and you're going to predict a 0 or 1 as an output from your machine learning model.

01:00:21.552 --> 01:00:23.552

Any questions on that so far?

01:00:23.552 --> 01:00:32.227

Here's another way to look at that. So we have our SST in the tropics. This is what I'm providing. And then we're asking, is it a 0, a La Nina, or an El Nino, a 1?

01:00:34.041 --> 01:00:48.041

Um, now you may be thinking that's a 2D image, and we've only been

talking about inputs that are 1D. So how do we deal with that in terms of artificial neural networks? The typical way is you basically just convert your 2D array into a 1D array by flattening it.

01:00:48.041 --> 01:01:04.969

This is what that would look like. So each grid point would be on your x-axis, your SST value is on your y-axis here. We have 31 latitudes and 180 longitudes, just based on the data set. I think we may have slightly changed the dataset, so this might not be the perfect analysis, but

01:01:05.139 --> 01:01:20.139

all that you need to know is that to determine the number, how big your input is, you just multiply the number of latitudes you have by the number of longitudes, and you'll see that we have 5,580 input values that we're putting into our artificial neural network.

01:01:21.097 --> 01:01:31.097

So each one of those input values is then mapped to a node in your machine learning model here, your ANN. So we end up having to learn a lot of weights

01:01:31.097 --> 01:01:40.321

just to, um... just to go from our original input of 5,000 plus to however big our first layer is going to be.

01:01:42.737 --> 01:01:52.737

Okay, so as I promised, here is an example of how you can determine the number of learnable parameters or trainable parameters in your machine learning model.

01:01:53.811 --> 01:02:12.811

So you can just do `model.summary` here. And it'll print this out. This is particular to Keras, which is what we're going to be using for this example. Keras is, I would argue, the simplest way to spin up on machine learning, but PyTorch is generally like the accepted machine learning algorithm in.

01:02:12.320 --> 01:02:27.320

If you're using Python. So in terms of spin up time, Keras is very quick in terms of long-term use and customization. I would recommend pytorch. Anyway, with that aside, in Keras, you can do `model.summary` here,

01:02:27.603 --> 01:02:43.603

and I'll just step through how we can get these number of parameters. So, as I mentioned, we have this 5,580 inputs in our machine learning model. Let's say we have a machine learning model that's 12 by 12 in terms of the hidden layers.

01:02:43.793 --> 01:02:50.793

Um, so the first hidden layer is 12 nodes, the second hidden layer is 12 nodes, and then our output is 2, so a 0 or a 1.

01:02:51.751 --> 01:03:07.751

So to get this 66,972 value here. Basically, what we're going to do is, as a reminder, each of those input of those 5,580 have to map to each one of those input nodes. So each one of the 12 nodes.

01:03:07.451 --> 01:03:15.451

And so in order to calculate that total number of weights that we need to learn, we do 5,580 times 12.

01:03:15.080 --> 01:03:31.080

That gets us to 66,960. As a reminder, we don't normally show with those bias terms in the diagrams. But we need to add these bias terms. So in order to get this 972

01:03:31.080 --> 01:03:37.118

we just add in the 12 bias terms here. That gives us the 66,972.

01:03:39.122 --> 01:03:50.122

So, if we want to do the 156th value here, we're going to take the 12 input values, and we're going to multiply that by 12, because we're doing 12 to 12. Now

01:03:50.123 --> 01:03:57.557

this leads to 144. And then we need to add the 12 bias terms, which will get us to 156.

01:03:58.449 --> 01:04:13.449

And then in terms of the last weights that we need to learn, we're going from 12 to 2, 12 times 2 is 24. We have those two bias terms that gets us to 26. So that's how you determine the number of learnable parameters for your artificial neural network.

01:04:15.325 --> 01:04:22.325

Which ended up being 67,154, with just a 12 by 12 model.

01:04:25.517 --> 01:04:37.517

Okay. So for this exercise, before we actually go into the code, I have 3 things that I'd like each of you to do during this exercise.

01:04:37.999 --> 01:04:54.999

Um, the first one is train a good model. I'll leave this definition up to you since you can change the lag in terms of your prediction. So you're... it's currently set to 3 months, so if your input, uh, your input will be 3 months before whether there's an El Nino or a La Nina in your output.

01:04:55.741 --> 01:05:03.741

Um, but you can change that to 12 months, see if you can get a good yearly prediction. Good will be different depending on how big your lead time is there.

01:05:03.973 --> 01:05:08.973

I think I define it as lag in the actual code. But same thing.

01:05:08.974 --> 01:05:17.656

Then I'm hoping that you guys can train a bad model. This may likely happen first, an example of a bad model is an overfit model.

01:05:17.703 --> 01:05:24.703

Another example of a bad model is a model that didn't learn anything. So try to find one of those.

01:05:24.431 --> 01:05:40.431

And then something that I also encourage is to try some non-standard values for hyperparameters. I think it's this is a very safe environment for you to try some crazy values, like a learning rate of 10, for example, or 100. Try many, many, many layers,

01:05:40.208 --> 01:05:46.208

try many, many, many nodes, see how that affects your training time, your performance.

01:05:46.540 --> 01:05:56.540

And maybe hypothesize before you actually train your model how your ANN might respond. If you break the code, you're doing a great job at trying things.

01:05:56.509 --> 01:06:14.509

So, that's great. All right. Lastly, some resources I recommend to get started if just in general in terms of machine learning. I've already mentioned these 3 blue one brown videos that I find phenomenal. But there's also a lot of resources online.

01:06:14.614 --> 01:06:31.614

Um, and then the last thing I want to mention is NCAR is actually creating an asynchronous machine learning tutorial that will be available to all on our website. This is in the process of being created, but hopefully once that's ready, I can send it all to you if you'd like to learn more.

01:06:33.355 --> 01:06:38.355

Okay, so Andrew, I think I'll pass it over to you now in terms of getting

01:06:42.558 --> 01:06:44.558

the code setup.

01:06:44.558 --> 01:06:46.813

Andrew: For sure. Sounds good, yeah. And then I'll pass it back to you to actually step through it, since I assume you have the notebook as well. Okay, cool.

01:06:46.814 --> 01:06:51.379

Kirsten: Yep, that sounds great.

01:06:51.380 --> 01:07:03.376

Andrew: Uh, okay. So I think, Inge, did you send out... I hope everyone here has a Google account of some variety. That's mainly the only prereq for this, since we're going to be using Google Colab, which is a really nice place to have some access to some compute resource.

01:07:03.376 --> 01:07:21.375

They also include as part of just the free tier, some really small GPUs, but even those small GPUs are going to be faster than most laptops that you're on, because, you know, it still does that kind of computation that we need to do for machine learning. One exception being if you're on a MacBook, there are-

01:07:21.361 --> 01:07:25.361

It does actually pretty well, um, for these kind of small networks as well.

01:07:25.590 --> 01:07:42.590

So I will share my screen, and kind of we'll just do this step by step. I'm going to drop a couple links in the chat. First of all, just to make sure and I, of course, closed the link that I needed to. So the first one is going to be this one here.

01:07:43.928 --> 01:07:48.928

Okay, so this is the actual Colab website that everyone should go to first.

01:07:48.928 --> 01:07:56.539

Um, and then the other one that I'm gonna drop before sharing my screen is if I can get back to my gist.

01:08:05.572 --> 01:08:10.572

Sorry if I was just here we go.

01:08:10.573 --> 01:08:22.282

Yeah. And this is the other link. So don't. You don't have to. I mean, you can click on this link, but for the second link, you're going to want to copy it into your... onto your clipboard. So let me now share screen.

01:08:29.239 --> 01:08:31.239

Oops.

01:08:31.710 --> 01:08:36.710

Sorry. Standard Zoom things.

01:08:37.982 --> 01:08:54.982

Here we go. Okay, so if you go to the `colab.research.google.com`, you're going to get immediately sent to something that looks like this, right? It's going to say, open a notebook, and because I've done other things before, we're going to see this here. What you're going to go to is you're going to go to this GitHub one.

01:08:55.255 --> 01:09:06.255

And then when it says, enter a GitHub URL or search by organization or user, you're just going to copy that link with the with this really strange set of letters and numbers into it.

01:09:07.672 --> 01:09:14.672

And then all you're gonna have to do after that, there's actually no button here, but you just hit enter, and it should open a notebook up for you.

01:09:15.505 --> 01:09:20.505

Dhamma: Andrew, I don't think you're you might be on the wrong screen.

01:09:24.769 --> 01:09:27.769

I I can see you go through that.

01:09:27.770 --> 01:09:28.045

Andrew: Oh, am I? Okay. Sure. I'm a professional. I've done this before.

01:09:30.225 --> 01:09:37.225

Dhamma: It's stuck on your ANN notebook. Yeah.

01:09:37.226 --> 01:09:38.489

Andrew: Uh, where? Oh, oh, it's on... okay, yeah, let me... other Vivaldi window.

01:09:41.999 --> 01:09:49.999

the not as clean one. Okay. Okay, let's go through that again. `Colab.research.google.com`.

01:09:52.736 --> 01:09:59.736

Great. So it'll pop up this this little pop up thingy that you see the examples, notebook, Google Drive, GitHub on the side.

01:10:00.674 --> 01:10:03.674

And you should just be able to copy and paste that.

01:10:03.967 --> 01:10:12.967

Uh, that link there up here. I don't think you'll have to say include private repos. I think that's fine. And after you just copy and paste it, just hit enter.

01:10:19.304 --> 01:10:35.304

Good to go? Okay, I got one good one from Inge. Wow, okay. You got fireworks. Okay, Dhamma, you're good to go. If there's anybody who's having extra problems, Inge, did you manage to figure out how to open a breakout room or no?

01:10:42.524 --> 01:10:44.524

Inge: Unfortunately not. Sorry.

01:10:44.525 --> 01:10:52.632

Andrew: Okay. Okay, sounds good. We can... we can pause and wait here for a little bit. Um, is there anybody... please raise your hand right now if there's anybody who didn't get to that part. So what it should look like after you do this is that it should just open up and look like something like this.

01:10:58.694 --> 01:11:07.694

Okay, Inge, for some reason, my Zoom controls also disappeared, so... Anybody look like they have a raised hand?

01:11:11.617 --> 01:11:17.617

Inge: Uh, nope, there's no raised hands at the moment. I've got a couple of thumbs up that went past, so...

01:11:17.617 --> 01:11:21.418

Kirsten: That went shockingly smoothly.

01:11:21.419 --> 01:11:27.876

Andrew: Cool, okay. All right. So, yeah. Yeah, I know, other than fumbling with Zoom. I took one for the team, guys. So, uh...

Kirsten: Yeah, yeah, thanks.

Andrew: All right, I'll pass it back to you, Kirsten, if you want to go ahead and do this.

01:11:27.521 --> 01:11:35.521

Kirsten: All right. Okay, cool. Is that big enough for everyone to see?

01:11:36.935 --> 01:11:39.935

I can make it bigger. Okay. It's good? Okay.

01:11:40.133 --> 01:11:56.133

All right. So. Just to walk through these notebooks, I'm assuming

people are generally familiar with Python, so I'm just going to skip that. This is importing libraries and what those libraries are.

01:11:57.082 --> 01:12:12.082

Um, these functions here are all functions to either get the data, pre-process the data, calculate the Nino 3.4, get figure out which the Nino 3.4 index is, split the data. You'll notice I will just mention that splitting the data is

01:12:12.654 --> 01:12:19.654

very basic, so if you try to do something crazy in terms of splitting the data, it will immediately fail.

01:12:19.752 --> 01:12:23.752

So just a note on that, if you want to create your own function, feel free.

01:12:24.901 --> 01:12:38.901

Um, standardizing the data, making it a categorical prediction problem. So saying anything that's greater than whatever your end. So magnitude is is a one. Anything less than is a 0. That's where that happens. Some plotting here.

01:12:40.217 --> 01:12:58.217

That's not interesting. Okay, so this is the where we're actually getting into defining our artificial neural network. I've just created this function that I've called define ANN. It takes in the number of hidden layers as a list where each of the list values are the number of nodes in that specific

01:12:58.217 --> 01:13:13.422

hidden layer. We have our input shape, which will be the size of your flattened SSTs. You'll have your output shape. This is a 2. You have an option of inputting a ridge penalty and a lasso penalty. It looks like.

01:13:13.712 --> 01:13:19.712

Um, here's your activation function. I have it set as the same for as.

01:13:20.053 --> 01:13:27.053

It'll be the same for how many ever... how many layers you use, it'll always be that activation function for each of the layers.

01:13:27.054 --> 01:13:33.884

And then a seed, a network seed, which is there in hopes that you can just reproduce your results as you train.

01:13:34.364 --> 01:13:43.364

Okay. So this is where Keras.input is how you define your input layer. So that's where we'd give it the shape of our input.

01:13:43.268 --> 01:13:55.268

Keras.layers.dense, dense layer is what we refer to as a hidden layer in an artificial neural network, which is why we use the term dense here.

01:13:55.320 --> 01:14:10.320

So our first layer is going to be the first, we're going to specify how many nodes are in that layer. So we're indexing our first layer in our list here. We define the activation function, and these are just some additional regularizers like

01:14:10.511 --> 01:14:23.511

L1 and L2, as we've already talked about. Then we're going to add in some more hidden layers here. So this will just loop through the rest of the hidden layers that if you add add in any more, it'll loop through those.

01:14:23.561 --> 01:14:40.561

Um... And then this has the same values that are up here. And then we have our output layer, which is also a dense layer. So we just have our output shape. Here's that activation function as softmax. We're only going to change.

01:14:40.304 --> 01:14:50.304

if this was a regression code, we'd likely make this linear instead of softmax. Or none, I think, is also an option, which means linear.

01:14:51.055 --> 01:15:02.055

And then this is where we actually create our Keras model. So Keras.model. We say our inputs are inputs and our output, our output layer. So inputs is up here defined here.

01:15:02.918 --> 01:15:09.918

And then our output layer is defined here. And then it will return your model.

01:15:10.693 --> 01:15:33.693

Okay, so that's a function that we'll pop back to once we get further down this this exercise here. I'm going to move a little bit faster, just given the time. So this is where we get SST data. This will plot your Nino 3.4 index. Here's where you can determine your lag or your lead, however you want to say it, in terms of the difference between your SST and the Nino 3.4

01:15:33.123 --> 01:15:40.123

index that you're predicting. Here's the ENSO magnitude set at 0.5. Feel free to change that.

01:15:40.332 --> 01:15:55.332

This will get your Nino 3.4 events. This will split your data. As I mentioned, this isn't a very sophisticated way to split your data. So right now this means that the first 70% are for training. The next 15 or

01:15:55.523 --> 01:16:10.523

15% is for validation, and the remaining 15 is for testing. You can. It has to be set up like that, unfortunately. But you could change it to like 0.9, and then 0.05 and 0.05, for example, you could do that as well.

01:16:10.912 --> 01:16:18.912

See how that changes things. All right. And then here's where we make it a categorical prediction problem.

01:16:19.127 --> 01:16:37.127

This is just more of some things you've already seen. Okay, so here's the train ANN section. This is where you determine what activation function you want to use. We have sigmoid, ReLU, tanH like I've already mentioned. Here's hidden layers. At this point. There's one hidden layer with a size of 4.

01:16:37.964 --> 01:16:44.964

Um, with 4 nodes in it. Um, if you did 10 comma 20, that's 2 hidden layers. First layer has 10 nodes, second layer has 20.

01:16:44.964 --> 01:16:57.398

We have our batch size. It's currently set to 4. You have your learning rate here currently set to 0.1, number of epochs is set to 20 right now. Ridge penalty is set to 0. We don't have one.

01:16:57.502 --> 01:17:02.502

Um, Optimizer can either be Ada or SGD here.

01:17:02.283 --> 01:17:09.283

And then your loss function is categorical cross entropy. This is standard for doing a categorical prediction problem.

01:17:09.132 --> 01:17:21.132

So this is where we call that define ANN. So we have number of hidden layers that we provide the input shape, the output shape, our activation function, and our ridge penalty.

01:17:21.473 --> 01:17:26.473

This is where we determine which optimizer we're using.

01:17:26.915 --> 01:17:42.915

This is where we actually compile our model, give it the optimizer, the loss function, and any metrics we want to track. And then here is actually where the training occurs. It's called ANN.fit kind of

similar to how we fit a line. We're now fitting whatever the AN learns.

01:17:42.849 --> 01:17:55.849

And this is where you provided your X train and Y train data, but also your validation data to evaluate its performance. You'll provide the number of epochs, batch size, and this verbose term is letting you know, um.

01:17:55.984 --> 01:18:03.984

How is... if you set it to zero, you will lose all of this printout. So if you're getting really annoyed with all of those printing out, you can set that to zero.

01:18:04.932 --> 01:18:20.932

Um, here's the number of parameters. Model.summary that I've already talked about. It automatically prints out. And then here is your training and validation lines with your loss, which is what we looked at when we were talking about overfitting. So.

01:18:20.933 --> 01:18:37.127

The training data is down here and the validation data is up here in terms of its performance. And then I find it easier to understand accuracy, which is just the number of correct predictions over all predictions. And so that's a better way for me to understand how well the model is doing.

01:18:38.477 --> 01:18:53.477

And then you can evaluate your machine learning model on your test data set after you've picked what model you like the most with your validation data set. You can then do these lines here, which will tell you how well you're doing on your La Nina and El Nino.

01:18:53.706 --> 01:19:08.706

and then you can do some composites of your correct predictions. If that's of interest. If you cannot find anything that seems to work for you, I do have an option. It doesn't mean it's the best option. That's cool. Whatever you just did there.

01:19:12.970 --> 01:19:14.970

?: What?

01:19:14.970 --> 01:19:21.448

Kirsten: Anyway. If you can't figure out something, then you can look at this solution, a solution to figure out what you might want to change, or you can ask us instead of looking.

01:19:21.539 --> 01:19:30.539

Okay? And with that. Have fun and let us know if you have any questions in the chat or verbally.

01:19:30.539 --> 01:19:32.123
Yeah.

01:19:35.409 --> 01:19:38.409
Dhamma: Um, are there breakout rooms, or...

01:19:38.409 --> 01:19:40.340
Kirsten: No, unfortunately, none.

01:19:40.341 --> 01:19:40.454
Dhamma: Okay.

01:19:50.971 --> 01:19:59.971
Andrew: And let's just confirm, if you, uh, can just thumbs up in general, like, if you can at least run through the notebook, like, run all the cells in the notebook and make sure it works.

01:19:59.971 --> 01:20:05.988
Cool.

01:20:05.989 --> 01:20:12.432
Inge: I don't know if I'm the only one, but I keep getting, like, a warning every time I do a new cell. Is that.

01:20:12.919 --> 01:20:15.919
Everybody else as well, or is it just me?

01:20:15.920 --> 01:20:20.881
Kirsten: What's the warning?

01:20:21.137 --> 01:20:22.917
Inge: It's that warning for the the not the it's a GitHub thing.

01:20:22.917 --> 01:20:30.930
Kirsten: Oh, yeah, just say, Okay, or whatever.

01:20:30.931 --> 01:20:32.568
Inge: Okay.

01:20:32.569 --> 01:20:34.351
Amber: Yeah, I just got that warning at the start, and then now it seems fine.

01:20:33.968 --> 01:20:36.968
Andrew: Inge, you want to share yours real quick and just see?

01:20:41.690 --> 01:20:48.690
Inge: Hold on... That's the right one. Yep. Okay.

01:20:49.148 --> 01:20:57.148

Cool. So if I try and run it. and then it asks to run anyway or.

Kirsten: Yeah, hit run anyway. Run anyway. Yeah.

01:20:57.260 --> 01:21:02.260

Inge: Okay. And then just... Okay, cool.

01:21:02.268 --> 01:21:05.268

Andrew: Yeah, it should just be in the 1st time, but yeah.

01:21:05.269 --> 01:21:11.428

Inge: Cool. Well, if there's anybody else, they know what to do now.

01:21:16.405 --> 01:21:27.405

Andrew: Like, I think this is definitely one of the things that I'm sure you've run into Kirsten. I've definitely run into anytime you run a tutorial, it is very unlikely you are the only person running into something strange.

01:21:31.860 --> 01:21:39.860

So it's always helpful when people are willing and have the courage to be like, yeah, something's not right. So...

01:21:39.861 --> 01:21:42.610

Kirsten: Absolutely.

01:21:54.009 --> 01:22:02.009

Andrew: Oh, and one thing I wanted to comment on, too, just call back to Amber, you know, so the hyperparameter question, right? Do you have an understanding of what

01:22:06.874 --> 01:22:09.874

we we would consider hyperparameters in this exercise?

01:22:09.874 --> 01:22:13.734

Amber: Um, all these things in box the train ANN box.

01:22:17.068 --> 01:22:32.068

But yeah, I guess I was thinking, because I was the hackathon we did, we were learning, talking about Gaussian process regression. And for that like the functional form that you choose to start with has a lot of hyperparameters. So it's so I was just trying to wrap my head around like.

01:22:32.464 --> 01:22:37.464

Um, it's interesting, like, now I've learned so even the number of nodes

01:22:37.514 --> 01:22:47.514

is a hyperparameter. So your architecture is defined then by some of these parameters that you choose.

01:22:48.103 --> 01:23:02.103

Andrew: I think Kirsten put it in a really good way when and during the lecture, right? Like the hyperparameters are not. They don't change over the course of a training over a training run right? They are fixed and constant and held constant through it.

01:23:02.870 --> 01:23:18.870

You can always find exceptional things, but I think that's a really good kind of, like, general idea of what you should think about there is that like they do. They're constant fixed over the course of a single training session. They do not get updated by that back propagation and optimization part of it that Kirsten was talking about.

01:23:19.415 --> 01:23:33.415

Amber: Right. With this parameter for the hidden layers, it started off as just being a single number, but it suggests that you can pick two numbers there and then it'll be the number of nodes.

01:23:34.292 --> 01:23:45.292

Um... like 10,20. Like, I'm just trying to wrap my head around, like, what it means to have, like, only one number in the hidden layers versus a number, comma number.

01:23:45.293 --> 01:23:55.030

Kirsten: Yes. Okay. So to clarify that. So the where if it's just a list with one number, that means there's one hidden layer. And in that case with 4 nodes.

01:23:55.031 --> 01:24:00.319

Amber: Okay.

01:24:00.319 --> 01:24:04.362

Kirsten: Yeah. And so the 10,2 is 2 hidden layers. First layer is 10. Or is it 20, 10?

01:24:08.170 --> 01:24:10.170

Andrew: 10 comma 20.

01:24:10.170 --> 01:24:10.983

Kirsten: No, 10. Okay, so the 1st layer is 10 nodes, and the second layer is 20 nodes.

01:24:17.735 --> 01:24:19.735

Okay.

01:24:19.736 --> 01:24:22.564

Andrew: So then you have a total of 4 layers in the network right? Because it's one input layer, the 2 hidden layers, and then the one output layer.

01:24:22.288 --> 01:24:28.288

Kirsten: Yeah, good clarification. and you can add way more than just 2. That was just.

01:24:28.547 --> 01:24:32.547

Amber: Is there some intuition into like how many you should need?

01:24:34.295 --> 01:24:36.268

Andrew: I mean, that is the dark arts of machine learning.

01:24:36.269 --> 01:24:43.829

Amber: That's where the wizardry comes in?

Andrew: Yeah.

01:24:43.829 --> 01:24:45.088

Dhamma: With two layers, you can think of it as, like, kind of a quadratic function.

01:24:51.090 --> 01:24:54.497

Amber: Oh.

01:24:54.497 --> 01:24:56.244

Dhamma: So the more layers is more complexity is the intuition I give myself.

01:25:02.338 --> 01:25:04.338

Yeah.

01:25:04.338 --> 01:25:04.823

Andrew: Uh, so Robin has a question. About hyperparameters.

01:25:04.824 --> 01:25:08.653

Kirsten: Yeah, I'm... I'm just gonna Google hyperparameter definition.

01:25:17.820 --> 01:25:20.820

Alright. Oh, yeah, go ahead, Dhamma.

01:25:20.820 --> 01:25:22.436

Dhamma: Oh, like, mathematically, I'm trained, like, trained more formally. A parameter is something that you estimate.

01:25:24.527 --> 01:25:27.527

Like a statistical model has parameters that you estimate.

01:25:27.880 --> 01:25:34.880

Um, in the, like, fitting process. And a hyperparameter is something that is not fitted by the fitting process.

01:25:41.075 --> 01:25:43.075

Kirsten: Perfect. That's what Google says.

01:25:43.076 --> 01:25:46.002

Dhamma: I don't know if that helped. Okay. But yeah, that's its statistical like, yeah, this all these terms like.

01:25:46.227 --> 01:25:59.227

Are kind of... Usually, corresponding with the, like, statistical estimation literature, but sometimes they're, like, a little fudged and weird. But in this case, I think it's pretty one-to-one.

01:26:12.708 --> 01:26:23.708

Inge: So just going back to Amber's point about like the hidden layers with the 10 to 20. So at the moment, like the set here is four. So that's pretty much it's just one layer.

01:26:23.780 --> 01:26:28.780

And if we wanted to make more, we would have to add that as well. Okay, great.

01:26:28.781 --> 01:26:30.786

Kirsten: You got it. Yep, one layer, 4 nodes. Yep.

01:26:30.688 --> 01:26:35.688

Inge: Cool. Oh.

01:26:35.688 --> 01:26:36.268

Andrew: Dhamma, do you want to try to take a stab at Diego's question in the chat?

01:26:47.613 --> 01:26:54.613

Dhamma: There's, like, conflicting... there's a lot of conflicting, like, literature on this, so...

01:26:54.613 --> 01:27:05.053

So, I'll start off with the fact that there is a fundamental theory... as fundamental of a theorem as you can get in neural network architecture, there is what is called the universal approximation theorem, which says that hypothetically, with one

01:27:05.678 --> 01:27:14.678

or with that any function can be approximated by a neural network if it has one hidden layer of

01:27:14.753 --> 01:27:26.753

infinite of large N , you know. So it can approximate any function.

What that large N is is also is an unknown question. Also,

01:27:26.711 --> 01:27:43.711

what that practically also means is that, you know, it comes to this question of, like, okay, do you want to go for a deep network, which is, like, more and more layers, or do you want to go for a wide network, which means that every layer has, like, a lot of neurons in it, right? So it's very... so, and that's what we call wide versus deep networks, right?

01:27:43.329 --> 01:28:02.329

Number of neurons per layer is the width of that layer, depth of a network is how many layers are kind of go through this like activation function kind of thing. So again, a lot of machine learning, like we there because of the scales of the problems and because of the nonlinear of the problem,

01:28:02.358 --> 01:28:18.358

there are very few things that we can actually prove from standard techniques. So this question that you're asking about number of parameters versus number of layers is actually not has not been proven in either way, right? For particular classes of problems,

01:28:18.987 --> 01:28:34.987

you can kind of make some kind of things, but those are very artificial problems. So, you know, this is the real magic of machine learning and engineers where it is, it is the intuition that you build up over time based on your knowledge of the problem, based on the knowledge of your data set, and also just brute force

01:28:34.466 --> 01:28:40.466

experimentation by playing with these things to figure out what is best.

01:28:41.394 --> 01:28:53.394

Kirsten: And sometimes now that machine learning has been in the field a lot longer, you might be able to find maybe a similar application and you could try to start from their architecture as a way to. To start.

01:28:55.801 --> 01:29:11.801

Andrew: And this is... that's actually a really good point, Kirsten, too, right? So when you're looking through the literature, one of the... if we think that we have like to name things like for proposals or things like that, the machine learning community has taken off on, like, naming things. So when you hear something like, oh, I used a ResNet model.

01:29:12.129 --> 01:29:27.129

Like to someone like Dhamma, to someone like Kirsten, someone like me,

we know exactly that there's a paper out there that showed this particular architecture and went to it. But there are so many model names out there, but if you hear something like that, like, oh, I use X, Y, or Z model.

01:29:28.013 --> 01:29:43.013

You can usually find the paper that was published that kind of talks about the architecture, talks about, like, you know, the sensitivity of some of these things, too. So, exactly what Kirsten was saying is there are so many models out there that have been designed.

01:29:43.948 --> 01:29:49.948

But they usually have a name associated with it. And so, yeah, there are some literature out there to kind of get a starting point.

01:29:51.863 --> 01:29:56.863

Dhamma: Yeah, um, I guess I'll add, in my experience with geospatial problems, like the

01:29:56.862 --> 01:30:03.862

broad strokes, so you're going to hear this term called inductive biases.

01:30:03.470 --> 01:30:09.470

And how do I explain that in a layman's way?

01:30:10.848 --> 01:30:19.848

The broad strokes of the broad design of the architecture, because right now we're just doing linear layers, is more important than the actual

01:30:19.943 --> 01:30:27.943

number of parameters in my experience. Right now we're kind of we're just fixed to this linear

01:30:27.261 --> 01:30:38.261

architecture, but when you start approaching more complex problems, you're going to consider things like transformers or ResNets or you know, things that have

01:30:38.820 --> 01:30:46.820

vastly different inductive biases. Yeah. What's a good way to explain an inductive bias, Andrew?

01:30:46.568 --> 01:30:56.568

Andrew: So I kind of classify, and maybe this is not the most accurate, but I kind of tend to talk about inductive bias as ways of.

01:30:58.727 --> 01:31:03.727

simplify, or making the problem easier to learn by adding structure

01:31:03.889 --> 01:31:19.889

to things. So, like, you know, like, for example, if you know that you want to take a derivative of something, right, you could learn that with these linear neurons, right? How to take a finite difference. It is just a mathematical thing, or there are things like, you know, convolutional neural network.

01:31:19.889 --> 01:31:31.547

Right, which are basically do what convolutions do and, like, apply certain weights that are learnable, that do the combinations of things as well. So you could learn... so a convolutional neural network.

01:31:31.528 --> 01:31:49.528

you know, it does just have a different mathematical operator that essentially is being performed on your input data. And so if you think that that kind of operation is needed for your work, then you know you could inject inductive bias into your machine learning

01:31:49.207 --> 01:32:05.207

model by choosing those specific layers. More advanced, like in like, you know, when we're talking about weather yesterday, right? Even like something like, you know, Nvidia as part of that NVIDIA's first forecast net model, one of the things that they pioneered, quote unquote,

01:32:05.253 --> 01:32:15.253

was this idea of a Fourier neural operator. So instead of working in real space, you know, these are neurons that know how to do a Fourier transform.

01:32:17.253 --> 01:32:33.253

Um, or equivalent of a Fourier transform in order to make the problem quote-unquote easier to solve. So again, like, you know, those are... and there are other ways you can do it, but it's, to me, the definition that I kind of work with, the working definition I use is anything that you do to the network

01:32:33.254 --> 01:32:39.571

to actually simplify the problem and/or guide down certain pathways of learning.

01:32:46.741 --> 01:32:49.741

I will also say that depending on who you talk to,

01:32:49.828 --> 01:32:55.828

whether inductive bias is good or not is definitely a matter of religious debate in the AI/ML community.

01:32:57.000 --> 01:33:11.000

I think more physical side of things, scientific side of things, we're like, yeah, we kind of want to do this, because we know that there's structure to our problems. Whereas if you talk to people who do, like, LLMs or computer vision, they tend to be very anti-inductive bias, but yeah.

01:33:12.391 --> 01:33:16.391

So don't be afraid if you post. Yeah, go ahead.

01:33:16.391 --> 01:33:20.436

Dhamma: I think it depends on how much. Yeah. Oh, I think it depends on how much inherent structure you add to your problem.

01:33:24.578 --> 01:33:27.578

Like an LLM is pretty unstructured, I think.

01:33:39.140 --> 01:33:46.140

Amber: So how do you keep track of all the tests you do? Do you just make like a spreadsheet and...

01:33:46.140 --> 01:34:05.880

Andrew: So there's so this is again, like, there's a lot of tooling that's kind of been brought up to do some of this stuff to track parameters and do the model tracking stuff like that, so... You know, especially if you're doing something, well, actually, Dhamma, why don't you talk about it? I think you know you probably have a better sampling of the AI/ML tools like weights and biases and things like that that are out there in the world.

01:34:07.911 --> 01:34:12.911

Dhamma: Yeah, um... Honestly, I haven't really used that recently.

01:34:19.511 --> 01:34:22.511

There are. Yeah, go for it, Kristen.

01:34:22.512 --> 01:34:34.886

Kirsten: I could say. There are these tools that are out there that allows you to do automated hyperparameter searches. And basically you specify like a range of values that you want it to test, and it'll go through and try to find the best architecture for you. And in that way you can.

01:34:34.398 --> 01:34:41.398

um, save off like what architectures you've tried using those types of.

01:34:41.187 --> 01:34:52.187

functions. But the simplest way is if you wanted to do it by hand, is what I've done before is put it in a Excel spreadsheet.

01:34:56.778 --> 01:34:58.778

Amber: Okay.

01:34:58.778 --> 01:35:00.630

Dhamma: I also do excel.

01:35:00.631 --> 01:35:01.625

Andrew: Yeah, no shame in a good Excel spreadsheet.

01:35:01.626 --> 01:35:01.800

Yeah.

01:35:01.800 --> 01:35:07.293

Amber: Yeah, because I...

01:35:07.293 --> 01:35:10.966

Kirsten: Typically, if you are publishing a paper, most people ask that you do like a formal hyperparameter search, which would probably be using one of these online tools.

01:35:17.043 --> 01:35:18.043

Amber: Oh, okay.

01:35:18.043 --> 01:35:28.507

Andrew: And you'll kind of see that also referred to sometimes. Well, sometimes you'll see those are referred to as like ablation studies, which is a very weird term that I did not was not aware of before machine learning. But yeah, that's kind of what they're talking about. Is that sensitivity aspect of it.

01:35:28.263 --> 01:35:40.263

Amber: Okay, okay, that's helpful. I've seen that word thrown around and then looks it up and you're like... I don't think that's what's meant. But yeah, that makes sense.

01:35:41.077 --> 01:35:47.077

Andrew: Also with probably another thing that's worthwhile to point out is that at some point you may want to compare

01:35:47.238 --> 01:36:02.238

the accuracy of your machine learning models, like, based on different networks, right? So, like, you owe you, there's 5 or 6 networks that you kind of think, you know, and the question over is, as Kirsten was saying, is like, what, what is... what does accuracy mean?

01:36:02.659 --> 01:36:14.659

There's one of the challenges in machine learning is that there's actually a lot of stochasticity. So when Kirsten was showing at the top of this that we're setting like a particular seed right?

01:36:14.762 --> 01:36:19.762

The optimization. the the your starting point,

01:36:20.787 --> 01:36:30.787

without even going to, like, stochastic methods of optimization, where you start out, if you just randomly initialize that, you may not get to that same minimum.

01:36:30.017 --> 01:36:47.017

And so... and that might mean that you are slightly less accurate, depending on the random number seed that you set from the beginning, your initial starting point. So, you know, the sensitivity to the random starting seed is actually something that is just best part of best practices when you're trying to compare

01:36:47.686 --> 01:37:00.686

networks nowadays, where you're like, oh, do, like, train this model 5 times with five separate seeds, and then you can get some sense of statistical power about whether, you know, two methods are different, or two architectures are better or not.

01:37:03.276 --> 01:37:10.276

Kirsten: Yeah, I was just going to throw up. Um, to help maybe visualize that.

01:37:10.470 --> 01:37:19.470

Um, and, like, a really simple case here. If you have like let's say you you didn't know.

01:37:19.148 --> 01:37:34.148

Let's say this was like a crazy space. If you randomly initialize your machine learning model, whatever parameters P that you randomly start with, you're going to end up in like a random spot on this like loss function here,

01:37:34.458 --> 01:37:51.458

is basically what we're saying. And so you... when you're training a machine learning model, you're this loss space is like a lot crazier than... than a 1D case. And so you could end up, like, over here, you could end up over here, and that may just lead into a different

01:37:55.289 --> 01:37:58.289

minima, depending on where you started.

01:37:58.289 --> 01:38:06.058

And so oftentimes, in addition to doing like a hyper parameter test, they also have you do like,

01:38:06.727 --> 01:38:15.727

like a bunch of seeds, random seeds, just to make sure that you that

your model is robust, depending on where you start within that space.

01:38:22.623 --> 01:38:28.623

Andrew: So I think, like, hopefully one of the things that everyone's abstracting from this conversation here is that there is

01:38:28.214 --> 01:38:43.214

a lot of fuzziness in AI and machine learning. There are no definite answers, right? If you're used to someone who's coming from theory, where you're like, I'm going to make this approximation, and then, you know, and then things flow from there, you know, and you get a definite answer,

01:38:43.463 --> 01:39:08.463

there's a lot of fuzziness in this, and so a lot of it, you know, you know, for Kirsten's career, Dhamma's career, my career, right, in terms of AI and ML, it's a lot of... kind of just guessing and checking and building up that intuition, understanding what parts of it help you move faster, right? Like, you know, um, you know, even the tooling around it, like NCAR, you know, if you look at some of the larger weather forecasting surrogate model training

01:39:08.322 --> 01:39:13.322

things out there, right? Part of that is just helping scientists explore

01:39:13.339 --> 01:39:18.339

these questions, these exact questions that everyone's been asking in the chat as well.

01:39:30.648 --> 01:39:42.648

Inge: Um, I have a question because I've heard that it's... that you almost have to also be careful of overtraining your model as well, that it's like you can almost go past

01:39:42.127 --> 01:39:46.127

like a converging point to a point that it

01:39:46.761 --> 01:39:54.761

it's sort of overdoes that, um... Yeah.

01:39:54.761 --> 01:40:03.443

Kirsten: I think what you've heard is probably about overfitting. So you just basically don't want to overfit to your training data. So if you train for too long, you're just going to learn your training data perfectly.

Inge: Okay. Okay. So it's related to that. Thank you.

01:40:08.518 --> 01:40:22.518

Dhamma: Yeah, I mean, you can also empirically select which part of the train where you, you could overtrain it, but then you could keep a log of your previous model versions and go back and just select one.

01:40:26.167 --> 01:40:30.167

Right. That's one technique.

01:40:30.167 --> 01:40:41.889

Andrew: Yeah, I think the the original interpretation of what you were saying, Inge, in terms of, you know, oh, maybe you're training too long. Typically, what that ends up doing is that you just don't get any benefit, so you're just.

01:40:41.712 --> 01:40:52.712

burning carbon and resource time, you know, to just kind of get to get to a plateau in the optimization. So, yeah.

01:40:52.820 --> 01:41:00.820

So there's things that you can think about doing that are called, like, early stopping, because, like, you know, in this notebook, uh, Kirsten is just specifying

01:41:00.569 --> 01:41:24.569

the number of epochs that it should train on. But there are things you can do, like, oh, exactly the same as many optimization things, like, if the percent improvement is too low for 5 iterations, then... Let's just stop because we're not getting anywhere. So that's what you would call, like, early stopping, which is more of what you were talking about, the spirit of the question that you were... or the spirit of the interpretation that you originally used.

01:41:28.080 --> 01:41:33.080

Dhamma: Yeah, I I can answer this question. Oh, Inge, are you? Do you have any more follow up questions?

01:41:34.540 --> 01:41:41.540

I could probably answer this question from Diego. So I think what you're talking about is like the endpoint.

01:41:42.022 --> 01:41:47.022

It depends on what you, how you define your dynamical system.

01:41:47.089 --> 01:41:52.089

Like is it the attractor you end up on?

01:41:52.518 --> 01:42:02.518

Um, which is, you know, the local minima that your optimization ends up on, or, like, the particular... optimization path that your algorithm goes through.

01:42:17.676 --> 01:42:22.676

Yeah, I would say it is sensitive to initial conditions like depending on where you start and all that.

01:42:25.597 --> 01:42:32.597

And so, as you can imagine, like, if you if you were to answer the second part of the question, can we treat the parameter space with some other mathematical tools?

01:42:33.142 --> 01:42:39.142

If you're doing hyperparameter optimization, like Bayesian techniques are oftentimes,

01:42:39.643 --> 01:42:54.643

you know, Bayesian optimization is is typically something that is very commonly used to kind of do that hyper parameter search. And then there's also, I think Kirsten was alluding to it yet in her talk, where it's just like you just do the full permutation of a bunch of ranges of parameters and just,

01:42:56.113 --> 01:43:00.113

you know, do a bunch of experiments, and just, and then track it.

01:43:09.075 --> 01:43:10.075

Does that answer the question, Diego?

01:43:17.328 --> 01:43:26.328

Andrew: So I'm kind of curious to see, now that you guys have been playing. Yes, it's definitely very fertile field for study, and it's a, you know, as you can imagine,

01:43:26.958 --> 01:43:42.958

training one model if it takes two weeks on, like, a thousand GPUs is already expensive, but doing this full permutation of hyperparameter studies is what really takes a lot of the training time, right? Like, you can imagine that's, like, probably it's very much the same as numerical model development.

01:43:42.959 --> 01:44:01.468

You're doing 10,000 runs to do to get a final configuration. Very similar. So those that two weeks now becomes 20,000 weeks of time, and... improving that process once we do it is a key part to making AIML kind of,

01:44:01.468 --> 01:44:01.477

Dhamma: Yeah.

01:44:05.590 --> 01:44:07.590

Andrew: just more efficient in general.

01:44:07.590 --> 01:44:13.600

Dhamma: Yeah, I remember showing up at NCAR and seeing inter-parameter ensembles. I was like, I know what this is. I've seen this.

01:44:17.104 --> 01:44:21.104

It's the same thing.

01:44:24.591 --> 01:44:28.591

I'm guessing most people here know what PPE is. Sorry.

01:44:28.591 --> 01:44:30.297

Andrew: So I'm going to... Go ahead.

01:44:30.298 --> 01:44:33.795

Dhamma: I'm guessing people here know what a PPE or inter parameter ensemble is.

01:44:35.301 --> 01:44:41.301

like that kind of. experiment. Yeah.

01:44:41.874 --> 01:44:45.874

Andrew: You want to give a quick overview? Because I saw Inge.

01:44:45.874 --> 01:44:49.036

Dhamma: Oh, no. Ah Kirsten's done this.

01:44:50.001 --> 01:44:53.001

What is... Oh, you haven't done this?

01:44:53.001 --> 01:45:05.317

Kirsten: I have not done a preferred perimeter ensemble. No, no, no. But it's basic, from my understanding is it's basically taking parameters and like slightly changing them and then running out the model and seeing what happens. So...

01:45:05.318 --> 01:45:06.596

Dhamma: Yeah.

01:45:06.596 --> 01:45:15.808

Andrew: Yeah, yeah. Yeah. So like, if you have like, you know, tuning like, you know, the easiest one in the ocean, right? It's like, what's the vertical mixing diffusivity? Right? You would change it from one to 1.05 or 1 to like 1.5 times 10 to the power negative 5 or something right? And then just see.

01:45:16.950 --> 01:45:22.950

What does Ocean heat uptake look like? What is the bias in my mixed layer depths? You know, that's exactly the same

01:45:22.950 --> 01:45:26.917

kind of question, right? We were... if you want to think about it in numerical modeling terms.

01:45:26.917 --> 01:45:36.722

That's, like, the hyperparameter search that we would be doing in order to find what we want to reduce the biases, the physical biases that we want to see.

01:45:42.417 --> 01:46:08.417

Yeah, so I was just gonna say, I'm kind of curious now that people have had 18 minutes to 18, 20-ish minutes to futz around with it and make mistakes and get messy, and all the good Miss Frizzle things. I don't know if that's a... Reference that people get anymore, Kirsten thank you. But yeah, be kind of curious to see if there's anyone who's kind of willing to just talk about the parameters they tried and the results that they got.

01:46:17.542 --> 01:46:19.542

If not, I might.

01:46:19.542 --> 01:46:28.841

Amber: I thought I thought it was interesting that I added the ridge pen and then it like just made it so like I had no false La Ninas, but like a whole bunch of false El Ninos. So.

01:46:28.794 --> 01:46:32.794

It was, yeah, so I guess it's rebalancing the

01:46:32.921 --> 01:46:46.921

weights, but that seemed to mess it up. So, and I guess my other thing that I've noticing too is like just how you have to check all these different things. Like, oh, it looks like it's doing good in this aspect. And then, oh, but it's actually not doing good.

01:46:49.803 --> 01:46:59.803

Andrew: As a as a, you know, person with numerical modeling background. It's there's some, I don't know, schadenfreude about watching AI machine learning model scientists being like,

01:46:59.723 --> 01:47:06.723

I can't maximize everything. It's like, yep, you're gonna have to just learn to deal with compromises.

01:47:06.724 --> 01:47:10.281

Amber: Yeah, compromise, yeah, you have to come to a compromise.

01:47:10.261 --> 01:47:18.261

Andrew: It's like you're never going to get a perfect regional model or perfect global model either, so... What are you trying for?

01:47:27.842 --> 01:47:34.842

Kirsten: I just want to thank Tessa for checking this zero ENSO magnitude example out for me.

01:47:47.261 --> 01:47:53.261

Andrew: Dhamma, do you want to take this one either from sort of experts kind of thing, or a.

01:47:53.261 --> 01:47:56.532

Dhamma: Merge two models that maximize different things. Oh, I see.

01:47:57.287 --> 01:48:11.287

Um... I haven't done this personally, but one thing I've done is I've trained using one objective or one setup, and then continued training the model on another objective,

01:48:11.882 --> 01:48:18.882

or another setup. And the... That... that's one way to optimize two objectives.

01:48:18.368 --> 01:48:25.368

Um, and kind of... it's one way I tune a machine learning model.

01:48:25.318 --> 01:48:32.318

Um... is by changing the training objective partway through. Yeah.

01:48:32.992 --> 01:48:40.992

There's also things like using an ensemble model. Uh, but... I haven't done that personally.

01:48:42.753 --> 01:48:47.753

But you know enough to kind of give the description of what an ensemble model is? Or do you want me to try to explain that?

01:48:53.033 --> 01:48:58.033

like better have Andrew do that, yeah.

01:48:58.034 --> 01:48:58.074

Or Kirsten, do you have any knowledge of ensemble modeling? I don't want to.

01:48:58.074 --> 01:49:01.928

Kirsten: I was sorry, I was not paying attention. I was answering a question in the chat.

01:49:01.929 --> 01:49:06.261

Dhamma: Oh, yeah, yeah. Do you have any experience in ensemble modeling that you want to kind of talk about?

01:49:10.220 --> 01:49:12.220

Kirsten: Like, normal models?

01:49:12.220 --> 01:49:13.246

Dhamma: No, machine learning models. AI models.

01:49:13.247 --> 01:49:20.024

Kirsten: Like how to make multiple? Usually I just perturb the seed to do that, or

01:49:19.958 --> 01:49:24.958

perturb the initial conditions similar to what we do with dynamical models.

01:49:25.551 --> 01:49:31.551

Yeah, the other option is. Oh, yep. Good.

01:49:31.552 --> 01:49:36.475

Dhamma: Oh, I think. All right. Oh, I think Diego was talking about two different maximization of minimization objectives.

01:49:37.292 --> 01:49:43.292

Like I think an ensemble of machine learning models. You're still optimizing the same.

01:49:43.292 --> 01:49:45.592

Kirsten: Oh, oh, I see. Ah.

01:49:45.592 --> 01:49:48.204

Dhamma: Except you just have like different versions of the model, for instance.

01:49:50.979 --> 01:50:08.979

Kirsten: This may be maybe this answers something. I'll give an example of something that I've done, which is taken 2 machine learning models, both ANNs, one predicts using ENSO information, and one predicts using MJO information. But they're predicting the same

01:50:08.419 --> 01:50:14.419

problem, just with different information, and then I linearly combined their predictions at the end

01:50:14.554 --> 01:50:24.554

to make a final prediction. So they're technically minimizing losses for the same problem, but using different information, and then we're combining them.

01:50:24.627 --> 01:50:29.627

But I can imagine you could just change that and combine it a different way.

01:50:33.565 --> 01:50:36.565

Yeah.

01:50:36.565 --> 01:50:51.340

Dhamma: I think at a, like, maybe a large bird's eye view, there are, like, techniques for selecting. If you have, like, a few different machine learning models, um, trained differently for different objectives, there's like some techniques for automatically

01:50:51.714 --> 01:50:55.714

training something to select which model to use as,

01:50:55.624 --> 01:51:10.624

Which model is more accurate? Which model like is expected to be more accurate or mix the mix the predictions together like you do with post-processing. NWP.

01:51:16.397 --> 01:51:29.397

Amber: But if you set your ENSO magnitude to zero, like I think you're, it's just nonsense, right? Because you're... You're just... that's going to change what's considered to be a true

01:51:29.080 --> 01:51:37.080

La Nina or El Nino, and then it's like, everything is either La Nina or El Nino, so it kind of doesn't make any sense.

01:51:37.918 --> 01:51:53.918

Kirsten: Well, yeah, I guess, so now you're, what you're asking it to do is let predict if there's going to be a warm SST anomaly or a cold SST anomaly in the Nino 3.4 index. And so it'll be interesting to look at the composites that you can make down below and see whether those composites

01:51:53.918 --> 01:51:58.036

look like El Ninos and La Ninas or not.

01:51:58.190 --> 01:52:06.190

Um, like traditional. I will say that we've... yeah. Yeah. Never mind. Forget that I continued.

01:52:05.785 --> 01:52:20.785

Andrew: Well, what I was gonna say is that if somebody is really feeling their Wheaties this morning and wants to extend the notebook, right, one of the really interesting things you could do is just say, this is a binary classification, but there's no reason why you can't do three classifications, so... or five, right?

01:52:26.504 --> 01:52:28.504

Amber: Hmm.

01:52:28.505 --> 01:52:34.215

Very high ENSO, like, strong ENSO, weak ENSO, neutral, low weak La Nina, strong La Nina. So you can generalize it to as many categories as you think exist.

01:52:34.637 --> 01:52:48.637

And, you know, for those of you who are really up on your ENSO literature, there are different flavors of ENSO, and so people have used AI techniques to try to identify the different flavors of ENSO as part of,

01:52:49.001 --> 01:52:58.001

yeah, you just have an SSTs and snapshot, but you don't necessarily know what flavor it is. So that classification problem is very classic in the... in the literature as well.

01:53:01.436 --> 01:53:07.436

Amber: Right? And now we have to account for climate change in our indices. Otherwise they don't work.

01:53:07.083 --> 01:53:17.083

Kirsten: This is a big conversation in the machine learning community is how we're going to take these machine learning models trained in our current climate and apply them to a future climate.

01:53:21.259 --> 01:53:24.259

Amber: Yeah, that's what I'm really interested in, too.

01:53:32.088 --> 01:53:43.088

Kirsten: Diego, I'm a little concerned about this comment. Does that mean that you are only predicting La Ninas and only predicting El Ninos? That could be one way that you're getting all your La Nina's correct

01:53:44.338 --> 01:53:51.338

and then all your El Nino events correct? Like it's decided that it's easier to predict all zeros than it is to learn anything

01:53:55.452 --> 01:53:57.452

within your data.

01:53:57.452 --> 01:53:58.294

Dhamma: Yeah, it depends on what the errors of each model are, right? Yeah.

01:53:58.294 --> 01:54:03.010

Kirsten: Yeah.

01:54:03.011 --> 01:54:08.461

Andrew: Or is that a hypothetical that you have a perfect model for

both events, for both event detection?

01:54:08.538 --> 01:54:16.538

Amber: Is one that's good for one and one that's good for the other. I think that... makes sense like with the the

01:54:16.105 --> 01:54:25.105

that ridge term really rebalanced things. And so I guess you can monkey around. But then, yeah, can you somehow combine those?

01:54:26.845 --> 01:54:34.845

Andrew: Yeah, so I mean, that that is that is the active question. And so, like, you know, you may have heard if you're following some of the literature.

01:54:34.823 --> 01:54:50.823

about like when deep seek the deep seek model came out for versus like the ChatGPT models and stuff like that that deep seek takes what's called like a mixture of experts approach. And so it's smaller models that like learn really good things, and then

01:54:50.174 --> 01:54:57.174

The math of experts usually has some kind of what Dhamma was talking about, some kind of, like.

01:54:58.066 --> 01:55:02.066

Another... yet another layer of machine learning model around it to try to select.

01:55:06.878 --> 01:55:08.878

Amber: Hmm.

01:55:08.879 --> 01:55:15.530

Andrew: And either select or route the query to each one of them. So, you know, so if you're interested in that, the literature space that you should look for is, um, yeah, machine learning ensembling methods, and a mixture of experts.

01:55:20.250 --> 01:55:32.250

Amber: Is there like different schools of thought on keeping things simple? Because, like, I would think, you know, we'd want to err on the side of keeping things more simple rather than making it more complicated.

01:55:36.555 --> 01:55:39.555

But what does the community think about that?

01:55:44.010 --> 01:55:45.590

Andrew: Depends on if you're asking Anthropic or if you're asking folks like us who have less resources.

01:55:47.698 --> 01:55:54.698

Amber: Or folks that don't care about using a lot of resources, because, I mean, even if you have a lot of resources doesn't mean you should use them.

01:56:10.232 --> 01:56:10.739

Andrew: So if you're... if you'll deal with me going to a like philosophical tangent, I actually am trying to write a position paper with philosophy professor that I know on this,

Amber: Oh.

Andrew: where, like the front like so frontier, what we call frontier AI models, right? So that's the chat Gpt is.

01:56:10.798 --> 01:56:16.798

the Anthropics, the Googles of the world, you know, have kind of moved to this place where they're trying to,

01:56:17.375 --> 01:56:33.375

to train a model or collection of models that have all the capability in the world, right? You can use it for coding, you can use it for art generation, you can use it for the image detection, all these kinds of things. So I think it kind of depends on your world view

01:56:33.409 --> 01:56:39.409

a little bit. So, and what we would like to argue for in the position paper is that if we look at

01:56:39.933 --> 01:56:55.933

ecosystems or just like groups of people, right? There is always a there is always a balance of generalist versus specialists. And you know, the types of models that the frontier labs are training are much more in that full generalist side of things. But if you look at

01:56:56.972 --> 01:57:11.972

resiliency, if you have just an ecosystem of generalists, they do not respond well right to perturbations. Neither do entire ecosystems are based on specialists, and you kind of need the mixing of them to actually get a robust set of complex interaction. So,

01:57:11.972 --> 01:57:24.980

if you ask me philosophically, I would say that the world is spending too much time on the frontier model, like that generalist kind of part of the world. I think as scientists, because we know that there's structural problems,

01:57:24.980 --> 01:57:31.594

we can probably get away. We do, we can, and we do get away with a

much smaller, simpler models that are fit for purpose.

01:57:37.539 --> 01:57:39.539

Amber: Hmm.

01:57:39.539 --> 01:57:40.391

Dhamma: Uh, I just sent a paper that came out recently in the chat that would be interesting to this conversation.

Amber: Hmm. Thanks.

01:57:41.337 --> 01:57:45.337

4 days ago. It's how fast this field moved.

01:57:50.656 --> 01:57:51.816

Kirsten: I just want to mention that it is noon, so.

01:57:55.222 --> 01:57:58.222

We're officially done.

01:58:00.842 --> 01:58:02.276

Amber: Thank you guys. This was really great.

01:58:02.276 --> 01:58:07.657

Andrew: Any parting questions from folks? Maybe take the temperature of folks if they want, you know, yet another

01:58:07.153 --> 01:58:18.153

you know, kind of in remote, but like kind of interactive kind of tutorial versus what Kirsten was talking about in terms of like having kind of your async course that you can kind of go through on your own.

01:58:21.792 --> 01:58:36.792

Inge: Yes, definitely. I guess if people can do thumbs up just showing whether they really appreciated this course and in terms and whether it was useful to be

01:58:37.295 --> 01:58:53.295

in-person, well, online in person, doing the course. Yeah, okay, great. That's it says a lot. But I think also the asynchronous does help definitely for if you can't join these workshops.

01:59:06.933 --> 01:59:08.997

Inge: It does. For everybody that's still online, before everybody disappears, please put on your cameras, like, we forgot to do the... the...

Andrew: The group shot.

Inge: what was it? The... Participants thingy.

